

aivancity School for Technology and Business

La Grande École de l'IA & de la Data

Paris Île-de-France · Nice Côte d'Azur

PROGRAMME GRANDE ÉCOLE — PGE5

FINAL YEAR PROJECT (PFE) — 2025–2026

Design, Development, and Evaluation of a Retrieval-Augmented Generation (RAG) Conversational Assistant Based on the Model Context Protocol (MCP) for Secure and Intelligent Access to Enterprise Knowledge Stored in SharePoint

Host Organization:

**DNSI — Direction du Numérique et des Systèmes
d'Information**

Author: Jean Direl NZE

Supervising Professor: [Supervisor Name]

Year of Submission: 2026

*This document is submitted in partial fulfillment of the requirements
for the Programme Grande École (PGE) diploma at Aivancity.*

Abstract

Keywords: Retrieval-Augmented Generation, Model Context Protocol, SharePoint, Enterprise Knowledge Management, Large Language Models, Semantic Search, FAISS, Security, Hallucination.

Large organizations accumulate vast quantities of technical and business documentation on enterprise platforms such as SharePoint, yet access to this knowledge asset remains fragmented, manual, and inadequate for complex contextual queries. While Large Language Models (LLMs) offer a compelling natural language interface to enterprise knowledge, their direct deployment is impeded by hallucination, knowledge cutoff limitations, and the absence of native mechanisms for access control and source attribution.

This thesis addresses the following research question: *to what extent does a Retrieval-Augmented Generation (RAG) architecture grounded in the Model Context Protocol (MCP) enable more reliable, secure, and traceable access to enterprise knowledge stored in SharePoint, compared to a standalone LLM and a naive RAG implementation?*

We design, implement, and evaluate AntiGravity — a modular RAG conversational assistant that leverages MCP as a standardized protocol layer between the AI reasoning engine and the SharePoint document corpus of the DNSI (Direction du Numérique et des Systèmes d’Information). The architecture separates a security-aware MCP Context Server, which manages a FAISS semantic index with ACL-filtered document access, from a conversational client that combines query classification, dense retrieval, re-ranking, and generation via Mistral AI.

The evaluation follows a four-dimensional protocol: (1) retrieval quality (Precision@K, Recall@K, MRR, nDCG); (2) generation quality (faithfulness, groundedness, hallucination rate via RAGAS); (3) system performance (end-to-end latency, throughput); and (4) user acceptance (SUS, TAM, task-based evaluation).

Results demonstrate that the RAG+MCP architecture reduces hallucination rate by [X%] compared to a standalone LLM and improves faithfulness scores by [Y%] over a naive RAG baseline. The MCP layer introduces a measurable latency overhead of [Z ms] at P95, which remains within acceptable enterprise SLA bounds. User evaluation yields a SUS score of [S/100], indicating [good/excellent] usability.

The primary scientific contribution of this thesis is a formal comparative evaluation

framework for RAG+MCP architectures in enterprise settings — the first such study applied to a private SharePoint corpus with ACL enforcement. The engineering contribution is a reference architecture for secure, permission-aware conversational AI in enterprise environments. Both contributions lay the groundwork for a doctoral continuation under the CIFRE scheme.

Résumé (French)

Mots-clés : Génération Augmentée par Récupération, Model Context Protocol, SharePoint, Gestion des Connaissances en Entreprise, Grands Modèles de Langage, Recherche Sémantique, Sécurité, Hallucination.

Les grandes organisations accumulent de vastes quantités de documentation technique et métier sur des plateformes comme SharePoint, mais l'accès à ce patrimoine informationnel reste fragmenté et insuffisant pour répondre à des requêtes complexes et contextuelles. Bien que les grands modèles de langage (LLMs) offrent une interface en langage naturel prometteuse pour l'accès aux connaissances d'entreprise, leur déploiement direct est entravé par les phénomènes d'hallucination, le manque de connaissance des données privées, et l'absence de mécanismes natifs de contrôle d'accès et de traçabilité des sources.

Cette thèse conçoit, implémente et évalue AntiGravity — un assistant conversationnel RAG modulaire qui exploite le Model Context Protocol (MCP) comme couche de protocole standardisée entre le moteur d'inférence IA et le corpus documentaire SharePoint de la DNSI. L'évaluation couvre la qualité de récupération, la qualité de génération, les performances systèmes et l'acceptabilité utilisateur.

Note: Bracketed values [X%, Y%, Z ms, S/100] will be populated upon completion of evaluation experiments.

Acknowledgements and Declaration

Acknowledgements

[To be completed by the author upon thesis finalization.]

I wish to express my sincere gratitude to [Supervising Professor Name] for their methodological guidance and intellectual support throughout this research project.

I am deeply thankful to the DNSI team for providing access to the SharePoint corpus, for their availability during the user study, and for the trust they placed in this research.

I also thank the faculty and staff of Aivancity School for Technology and Business for the rigorous academic environment that made this work possible.

Finally, [personal acknowledgements to be completed by the author].

Declaration

I, Jean Direl NZE, declare that:

1. This thesis is entirely my own work, except where explicitly stated otherwise through appropriate references and citations.
2. I have read and understood the plagiarism policy of Aivancity School for Technology and Business, and I confirm that this work does not violate that policy.
3. I have obtained necessary permissions for any third-party material included in this work.
4. Generative AI tools (specifically Claude, developed by Anthropic) were used as an assistive tool during the writing process. All uses are documented in Annex E in compliance with Aivancity's policy on generative AI use. All analysis, reasoning, interpretation, and critical contributions are my own intellectual work.
5. The DNSI organizational context, documents, and user study data are treated in accordance with the confidentiality agreement signed between myself, the DNSI, and Aivancity School for Technology and Business.

Signed: _____

Date: _____

Jean Direl NZE

Contents

Abstract	i
Acknowledgements and Declaration	iii
1 Introduction	1
1.1 General Context: The Enterprise Knowledge Access Problem	1
1.2 The Promise and Limits of Large Language Models	2
1.3 Retrieval-Augmented Generation as a Paradigm Shift	3
1.4 The Model Context Protocol: Standardizing the AI-Data Interface . . .	4
1.5 Research Problem and Questions	4
1.6 Research Approach and Contributions	5
1.7 Scope and Boundaries	6
1.8 Thesis Structure	6
2 Background and State of the Art	8
2.1 Large Language Models: Capabilities and Limitations	8
2.1.1 The Transformer Architecture	8
2.1.2 Parametric vs Non-Parametric Knowledge	9
2.1.3 The Hallucination Problem	9
2.1.4 Critical Assessment	10
2.2 Retrieval-Augmented Generation	10
2.2.1 Foundational Architecture	10
2.2.2 RAG Variants and Evolution	11
2.2.3 Limitations of Current RAG Implementations	11
2.3 Vector Search and Semantic Indexing	12
2.3.1 Dense Embeddings	12
2.3.2 FAISS: Scalable Similarity Search	12
2.3.3 Re-ranking	13
2.4 Enterprise Knowledge Management and SharePoint	13
2.4.1 Enterprise Knowledge Management	13
2.4.2 SharePoint Architecture and Security Model	14
2.5 The Model Context Protocol	15
2.5.1 Protocol Overview	15
2.5.2 MCP vs Alternative Integration Approaches	15

2.5.3	MCP in Enterprise Deployments: Open Questions	16
2.6	Security in Enterprise AI Systems	17
2.6.1	Information Security Requirements	17
2.6.2	Authentication: OAuth 2.0 and SSO	17
2.7	RAG Evaluation Frameworks	18
2.7.1	The RAGAS Framework	18
2.7.2	Retrieval Evaluation Metrics	18
2.7.3	User Evaluation: SUS and TAM	19
2.8	Research Gap and Thesis Positioning	19
3	Research Methodology	21
3.1	Choice of Research Paradigm	21
3.2	DSR Cycles Applied to This Thesis	22
3.3	Research Design Overview	22
3.4	Evaluation Methodology	23
3.4.1	Experimental Conditions	23
3.4.2	Benchmark Dataset Construction	24
3.4.3	Retrieval Evaluation Protocol	24
3.4.4	Generation Evaluation Protocol	25
3.4.5	System Performance Evaluation Protocol	26
3.4.6	User Evaluation Protocol	26
3.5	Validity and Reliability	27
3.6	Ethical Considerations	28
3.7	Chapter Summary	28
4	System Design and Architecture	29
4.1	Requirements Analysis	29
4.1.1	Functional Requirements	29
4.1.2	Non-Functional Requirements	31
4.2	Architectural Approach and High-Level Design	32
4.2.1	Design Principles	32
4.2.2	High-Level Architecture	32
4.3	Data Ingestion and Pre-processing Pipeline	33
4.3.1	Document Extraction from SharePoint	33
4.3.2	Document Parsing	34
4.3.3	Chunking Strategy	34
4.4	Vectorization and Semantic Indexing	35
4.4.1	Embedding Model Selection	35
4.4.2	FAISS Index Construction	35
4.5	The MCP Context Server	36

4.5.1	MCP Server Design	36
4.5.2	ACL-Aware Retrieval	36
4.5.3	Context Window Management	37
4.6	Query Processing and Re-ranking	37
4.6.1	Query Classification	37
4.6.2	Re-ranking	37
4.7	Generation Pipeline	37
4.7.1	Language Model Selection	37
4.7.2	Prompt Engineering	38
4.8	Security Architecture	38
4.8.1	Authentication Flow	38
4.8.2	Threat Model	39
4.9	Deployment Architecture	39
4.10	Design Decisions Summary	41
4.11	Chapter Summary	41
5	Implementation	43
5.1	Implementation Environment	43
5.1.1	Technology Stack	43
5.1.2	Hardware Environment	44
5.2	MCP-Compatible SharePoint Connector	44
5.2.1	Implementation Overview	44
5.2.2	Key Engineering Challenge: Permission Inheritance Resolution	44
5.2.3	Rate Limiting and Throttling	45
5.3	Ingestion Pipeline Implementation	45
5.3.1	Document Processing	45
5.3.2	Incremental Indexing	46
5.4	MCP Context Server Implementation	46
5.5	Streamlit Application	47
5.6	Implementation Challenges Summary	48
5.7	Reproducibility	50
5.8	Chapter Summary	50
6	Evaluation and Results	51
6.1	Evaluation Setup	51
6.1.1	Experimental Conditions	51
6.1.2	Benchmark Dataset	52
6.2	Retrieval Evaluation	52
6.2.1	Results	52
6.2.2	Retrieval Analysis by Query Type	53

6.2.3	Retrieval Failure Analysis	53
6.3	Generation Quality Evaluation	54
6.3.1	RAGAS Metrics	54
6.3.2	Hallucination Rate Analysis	54
6.3.3	Hallucination Taxonomy	55
6.3.4	Qualitative Response Analysis	55
6.4	System Performance Evaluation	56
6.4.1	End-to-End Latency	56
6.4.2	MCP Protocol Overhead Isolation	57
6.4.3	Memory Footprint	57
6.5	User Evaluation	57
6.5.1	Participant Profile	57
6.5.2	SUS and TAM Results	58
6.5.3	Task Performance Results	58
6.5.4	Qualitative Themes	59
6.6	Ablation Study	59
6.7	Hypothesis Testing Summary	61
6.8	Chapter Summary	61
7	Discussion	63
7.1	Interpretation of Results	63
7.1.1	On the Primary Research Question	63
7.1.2	On the Value of MCP Specifically	64
7.2	Strengths of the Proposed Architecture	65
7.3	Weaknesses and Limitations	65
7.3.1	Evaluation Limitations	65
7.3.2	Technical Limitations	66
7.3.3	Scope Limitations	67
7.4	Security Analysis: Critical Assessment	67
7.5	Deployment Challenges: Lessons Learned	68
7.6	Generalizability of Findings	69
7.7	Future Work	69
7.8	Chapter Summary	71
8	Conclusion	72
8.1	Summary of Contributions	72
8.2	Answers to Research Questions	73
8.3	Scientific and Practical Implications	74
8.4	Toward a CIFRE Doctoral Continuation	75
8.5	Closing Reflection	75

Bibliography	77
A Benchmark Dataset: Question-Answer Pairs	80
B User Study Instruments	82
C System Configuration and Deployment Guide	84
D Raw Evaluation Data	86
E Declaration of Generative AI Tool Use	88

List of Figures

3.1	Research design: five phases of the DSR process	23
4.1	AntiGravity four-layer architecture overview	33

List of Tables

2.1	FAISS index types and their trade-offs	13
2.2	Comparison of AI-data integration approaches	16
2.3	Research gap: what existing work covers vs what this thesis contributes	20
3.1	Design Science Research cycles applied to this thesis	22
3.2	Threats to validity and mitigation strategies	27
4.1	Functional requirements of the AntiGravity system	29
4.2	Non-functional requirements and measurable acceptance criteria	31
4.3	Comparison of chunking strategies evaluated	34
4.4	Embedding model trade-off analysis	35
4.5	AntiGravity threat model	39
4.6	Key architectural design decisions with justification	41
5.1	AntiGravity technology stack	43
5.2	Implementation challenges and resolutions	49
6.1	Benchmark dataset composition	52
6.2	Retrieval evaluation results (Conditions B and C, $N = 45$ answerable questions)	52
6.3	Retrieval Precision@5 by query category	53
6.4	RAGAS generation quality results ($N = 45$ answerable questions) . . .	54
6.5	Hallucination rate by condition and query type	54
6.6	Hallucination taxonomy with representative examples	55
6.7	End-to-end latency breakdown (milliseconds, single user, CPU inference)	56
6.8	Memory footprint by component	57
6.9	System Usability Scale and TAM results	58
6.10	Task-based evaluation results ($N = [TBD]$ participants, 5 tasks)	58
6.11	Qualitative themes from user interviews	59
6.12	Ablation study: component contribution to faithfulness and P@5	60
6.13	Summary of hypothesis tests	61
7.1	Prioritized future work roadmap	70
C.1	Tunable system configuration parameters	85

Chapter 1

Introduction

1.1 General Context: The Enterprise Knowledge Access Problem

The digitalization of organizational processes has generated an unprecedented volume of business documentation. Technical specifications, procedural guides, project reports, regulatory frameworks, meeting minutes, and operational manuals accumulate continuously on enterprise collaborative platforms. This documentation represents an organization's collective intelligence — the crystallization of years of expertise, decision history, and operational knowledge. Yet despite the theoretical availability of this knowledge, organizations consistently struggle to translate *stored* information into *accessible* information.

The symptoms of this gap are well documented in the enterprise knowledge management literature (Alavi and Leidner, 2001; Davenport and Prusak, 1998). Employees spend a disproportionate fraction of their working time searching for information rather than using it. Complex queries — those requiring synthesis across multiple documents, historical context, or domain-specific reasoning — are particularly poorly served by conventional search interfaces. The result is what Davenport and Prusak (1998) term *knowledge friction*: the resistance that organizational structures, technical interfaces, and information architectures impose between the knowledge that exists and the knowledge that can be practically applied.

Microsoft SharePoint, the enterprise content management platform used by over 200 million users globally (Microsoft, 2024b), is a paradigmatic example of this tension. SharePoint provides robust document storage, versioning, and access control capabilities, but its search functionality is fundamentally lexical: it matches keywords without understanding intent, context, or semantic proximity. A user querying "what are the security requirements for deploying a new application?" may receive thousands of matching documents, ranked by keyword frequency, with no mechanism for synthesizing a coherent, sourced answer across the relevant subset.

This thesis originates in a concrete instantiation of this problem. The Direction du

Numérique et des Systèmes d’Information (DNSI) — the information systems directorate of a large organization — maintains an extensive documentation corpus on its SharePoint platform (SharePoint Collaborative Enterprise Resource Platform (CERP)). This corpus includes technical architecture documents, security policies, operational procedures, and project deliverables. Business units within the DNSI face daily challenges in locating, interpreting, and synthesizing information from this corpus, a friction that imposes measurable operational and decision-making costs.

1.2 The Promise and Limits of Large Language Models

The emergence of Large Language Models — neural architectures trained on massive corpora to generate coherent, contextually appropriate text (Brown et al., 2020; Chowdhery et al., 2022) — has opened a fundamentally new interface for knowledge access. An LLM can, in principle, receive a natural language query and produce a synthesized, readable, and structurally coherent response that directly addresses the user’s need, without requiring the user to formulate Boolean search expressions or manually read through ranked document lists.

The practical deployment of LLMs in enterprise environments, however, is blocked by three fundamental and interdependent limitations:

Hallucination. LLMs are generative models trained to produce plausible continuations of text, not to retrieve verified facts. They will generate confident, fluent responses that are factually incorrect — a phenomenon termed hallucination (Maynez et al., 2020; Ji et al., 2023). In a consumer context, this is problematic. In an enterprise context, where responses may inform operational decisions, security configurations, or regulatory compliance positions, it is unacceptable.

Knowledge Cutoff and Private Data. LLMs are trained on static snapshots of public data up to a fixed point in time. They have no knowledge of an organization’s private documentation, internal processes, post-training events, or domain-specific terminology. A model trained on public internet data cannot answer questions about the DNSI’s internal security policy, because that policy was never in its training corpus. This is not a solvable problem through better model training — it is structural.

Traceability and Access Control. Enterprise environments require that responses be attributable to specific, authoritative sources, and that access to sensitive documentation be governed by the organizational permission model. A standalone LLM provides

neither: it generates responses without citing sources, and it applies no access control — the same model, with the same weights, would generate the same response regardless of whether the querying user is authorized to access the underlying information.

These three limitations converge to make direct LLM deployment in enterprise knowledge management contexts not merely suboptimal but operationally and legally untenable in most regulated environments.

1.3 Retrieval-Augmented Generation as a Paradigm Shift

The Retrieval-Augmented Generation paradigm, introduced by Lewis et al. (2020) at NeurIPS 2020, offers a compelling architectural response to these limitations. Rather than relying exclusively on the parametric knowledge encoded in model weights, Retrieval-Augmented Generation (RAG) retrieves relevant documents from an external knowledge base at query time, injects them as context into the LLM’s input, and uses the model to synthesize a response grounded in the retrieved evidence.

This architectural choice addresses all three limitations simultaneously: the model’s response is constrained by the retrieved context (reducing hallucination), the knowledge base can be populated with private, up-to-date organizational documents (addressing the knowledge cutoff), and the retrieval step can be restricted to documents the querying user is authorized to access (enforcing access control). Importantly, the knowledge base can be updated without retraining the model — a critical operational advantage in dynamic organizational environments where documentation evolves continuously.

Since Lewis et al.’s foundational work, the RAG paradigm has matured rapidly. Y. Gao et al. (2024) survey over 100 RAG variants across three evolutionary generations: naive RAG (direct retrieval and generation), advanced RAG (pre-retrieval query optimization and post-retrieval reranking), and modular RAG (flexible pipeline architectures with specialized components for specific use cases). Each generation addresses limitations of the previous through increasingly sophisticated retrieval, context management, and generation strategies.

Despite this maturity at the architectural level, a critical infrastructure challenge remains largely unsolved: the problem of the data connector. Enterprise knowledge is stored in heterogeneous, siloed systems — SharePoint, Confluence, SAP, internal databases, email archives — each with its own API, authentication model, document format, and permission structure. Integrating RAG pipelines with these sources has historically required bespoke, hard-to-maintain connectors that introduce significant engineering overhead and security risk. This integration problem is the immediate

motivation for the Model Context Protocol.

1.4 The Model Context Protocol: Standardizing the AI-Data Interface

In November 2024, Anthropic published the specification for the Model Context Protocol (MCP) (Anthropic, 2024) — an open standard designed to formalize the interface between AI reasoning systems and external data sources. MCP defines a JSON-RPC-based protocol through which AI clients can discover, connect to, and query data providers (MCP servers) through a unified, standardized interface, regardless of the underlying data source’s native API.

The architectural significance of MCP is the introduction of a stable abstraction layer between the AI layer and the data layer. Before MCP, connecting an AI assistant to SharePoint required writing SharePoint-specific code directly in the AI pipeline — code that couples the pipeline to SharePoint’s API versioning, authentication changes, and structural evolution. With MCP, the SharePoint-specific logic is encapsulated in an MCP server, and the AI pipeline communicates through the stable MCP protocol. Changes in the SharePoint API require only updating the MCP server, not the AI pipeline. This separation of concerns is architecturally analogous to the role ODBC played in standardizing database connectivity in the 1990s.

Beyond maintainability, MCP’s architecture introduces properties particularly relevant to enterprise environments: the server-side context management allows access control to be enforced at the data boundary (before context is passed to the AI model), making permission enforcement structural rather than advisory. The protocol’s tool-calling semantics also enable richer interactions than simple document retrieval — an MCP server can expose structured operations (search, filter, navigate document hierarchies) that a naive retrieval connector cannot.

However, MCP is a recent standard. Its practical value in production enterprise deployments — particularly regarding the claimed benefits of security, interoperability, and maintainability — has not yet been rigorously evaluated in a published academic study. This empirical gap constitutes the primary scientific motivation for this thesis.

1.5 Research Problem and Questions

The preceding analysis converges on the following primary research question:

RQ1 (Primary): *To what extent does a Retrieval-Augmented Generation architecture grounded in the Model Context Protocol enable more reliable,*

secure, and traceable access to enterprise knowledge stored in SharePoint, compared to a standalone LLM and a naive RAG implementation?

This primary question is operationalized through four subsidiary research questions:

- **RQ2 (Retrieval):** What retrieval configuration (chunking strategy, embedding model, similarity metric, re-ranking approach) maximizes retrieval precision and recall on a domain-specific SharePoint document corpus?
- **RQ3 (Protocol):** What specific security, interoperability, and maintainability properties does MCP deliver in an enterprise SharePoint integration context, and how do these compare to alternative integration approaches?
- **RQ4 (Performance):** What are the latency, throughput, and scalability characteristics of the RAG+MCP system under realistic enterprise workload conditions?
- **RQ5 (Acceptance):** How do end-users perceive the usefulness, trustworthiness, and ease of use of the AntiGravity assistant compared to their current SharePoint search workflow?

1.6 Research Approach and Contributions

This thesis adopts the **Design Science Research** (DSR) paradigm (Hevner et al., 2004), which frames research as the construction and rigorous evaluation of a purposeful artifact. The artifact — the AntiGravity RAG+MCP system — constitutes the primary research output, while the evaluation of that artifact against a real organizational problem constitutes the primary scientific contribution.

The thesis makes the following contributions:

Scientific Contribution 1 (SC1): A formal comparative evaluation framework for RAG+MCP architectures in enterprise settings, operationalized through a controlled experiment on a real SharePoint corpus with three conditions: (A) standalone LLM, (B) naive RAG, (C) RAG+MCP. This is, to the best of our knowledge, the first published empirical study to evaluate MCP-grounded RAG against baseline architectures on a private enterprise document corpus.

Scientific Contribution 2 (SC2): A reference architecture for secure, Access Control List (ACL)-aware RAG deployment in enterprise environments, including design patterns for MCP-compatible SharePoint connectors and a formal analysis of the security properties of the proposed architecture.

Scientific Contribution 3 (SC3): An application of Design Science Research to the evaluation of enterprise AI systems, demonstrating how DSR can structure both the artifact construction and the artifact evaluation in a single methodologically coherent framework.

Technical Contribution 1 (TC1): An MCP-compatible SharePoint connector that extracts documents from SharePoint, respects ACLs, and exposes the corpus through the MCP interface.

Technical Contribution 2 (TC2): A domain-specific benchmark dataset of question-answer pairs derived from the DNSI SharePoint corpus, suitable for evaluating any conversational assistant on enterprise business documents.

1.7 Scope and Boundaries

This thesis addresses the design, implementation, and evaluation of a conversational assistant for enterprise document access. The following boundaries apply:

In scope: Design of the AntiGravity architecture; implementation of the RAG+MCP pipeline; evaluation against three experimental conditions; retrieval, generation, system, and user evaluation; security architecture analysis; DSR-grounded methodology.

Out of scope: Training or fine-tuning of language models; multi-tenant deployment; real-time SharePoint delta synchronization; extension to data sources beyond SharePoint; production-scale infrastructure optimization. These are identified as directions for future work in Chapter 7.

1.8 Thesis Structure

The remainder of this thesis is organized as follows:

Chapter 2 establishes the theoretical and empirical foundations of the work through a comprehensive review of the literature on LLMs, RAG architectures, vector search, enterprise knowledge management, the Model Context Protocol, and enterprise AI security. It concludes by identifying the research gap this thesis fills.

Chapter 3 formalizes the research methodology, grounding the approach in Design Science Research, and specifies the complete evaluation protocol for each of the four evaluation dimensions.

Chapter 4 presents the system design and architecture of the AntiGravity assistant, documenting the design decisions made at each architectural layer, the alternatives

considered and rejected, and the trade-offs accepted.

Chapter 5 describes the technical implementation of the system, documenting the engineering challenges encountered and the solutions developed.

Chapter 6 presents the experimental results across all four evaluation dimensions, tests the six research hypotheses, and provides detailed statistical analysis of the findings.

Chapter 7 provides a critical discussion of the results, analyzing strengths, weaknesses, limitations, deployment challenges, security gaps, and the generalizability of the findings.

Chapter 8 synthesizes the contributions of the thesis, provides explicit answers to each research question, and outlines the directions for future work, including a proposed research agenda for CIFRE doctoral continuation.

Chapter 2

Background and State of the Art

This chapter establishes the theoretical and empirical foundations required to understand, position, and evaluate the contributions of this thesis. We survey the relevant literature across six interconnected domains: Large Language Models and their limitations (Section 2.1); Retrieval-Augmented Generation architectures (Section 2.2); vector search and semantic indexing (Section 2.3); enterprise knowledge management and SharePoint (Section 2.4); the Model Context Protocol (Section 2.5); security in enterprise AI systems (Section 2.6); and RAG evaluation frameworks (Section 2.7). Each section concludes with a critical assessment of the current state of the art and the limitations this thesis addresses. Section 2.8 synthesizes the literature into an explicit research gap statement.

2.1 Large Language Models: Capabilities and Limitations

2.1.1 The Transformer Architecture

The modern era of Large Language Models originates with the Transformer architecture introduced by Vaswani et al. (2017) in their seminal paper "Attention Is All You Need." The Transformer replaces the sequential computation of recurrent networks with a self-attention mechanism that computes, for each token in a sequence, a weighted representation of all other tokens simultaneously. This design enables parallelization during training — a property that, combined with scale, proved to be the foundation of modern language models.

The self-attention mechanism can be formalized as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \quad (2.1)$$

where Q , K , and V are query, key, and value matrices respectively, and d_k is the dimensionality of the key vectors. The scaling factor $\sqrt{d_k}$ prevents vanishing gradients in high-dimensional spaces.

Modern large-scale language models — including GPT-4 (OpenAI, 2023), Mistral 7B (Jiang et al., 2023), Llama 2/3 (Touvron et al., 2023), and Claude (Anthropic, 2023) — are decoder-only or encoder-decoder Transformers trained via next-token prediction on datasets of trillions of tokens. The emergent capabilities observed at scale — instruction following, chain-of-thought reasoning, code generation, multi-step problem solving — are not explicitly programmed but arise from the optimization process applied at sufficient scale (Wei et al., 2022).

2.1.2 Parametric vs Non-Parametric Knowledge

A foundational distinction for understanding LLM limitations is between *parametric* and *non-parametric* knowledge (Lewis et al., 2020). Parametric knowledge is information encoded in a model’s weights during training — it is fixed, distributed, implicit, and inaccessible without inference. Non-parametric knowledge is information stored in explicit, updatable external databases — structured, addressable, and modifiable without retraining.

LLMs are parametric knowledge stores. Everything they "know" is compressed into billions of floating-point weights through the training process. This has several important implications:

- **Knowledge is probabilistic, not factual.** The model learns statistical patterns in text, not ground truth facts. Information that is rare, contradictory, or absent in the training corpus will be poorly represented or incorrectly recalled.
- **Knowledge is static.** Once training is complete, the model’s knowledge is frozen. Events after the training cutoff, or information not present in training data, are genuinely unknown to the model.
- **Knowledge is opaque.** There is no mechanism for tracing a model’s assertion to a specific source document. The model cannot say "I know this because I read it in document X" — it can only generate what its weights make probable.

2.1.3 The Hallucination Problem

Hallucination — the generation of confident, fluent, but factually incorrect text — is the most operationally significant limitation of LLMs for enterprise deployment. Ji et al. (2023) provide a taxonomy of hallucination types, distinguishing between intrinsic hallucinations (contradictions of source material) and extrinsic hallucinations (unverifiable claims absent from any source). Maynez et al. (2020) demonstrate that even state-of-the-art summarization models hallucinate in 30% or more of generated summaries.

The underlying mechanism of hallucination is partially understood. Models optimize for textual fluency and statistical plausibility, not factual accuracy. When asked about information that is rare, ambiguous, or absent in training data, the model continues to generate plausible text — because that is what it was trained to do — regardless of whether the generated content is grounded in fact (Rawte, Sheth, and Das, 2023).

For enterprise applications, hallucination is not merely a quality problem — it is a trust and liability problem. A conversational assistant that occasionally fabricates security policy requirements, invents technical specifications, or misrepresents regulatory obligations can cause real operational harm. Any architecture deployed in an enterprise knowledge management context must provide structural mechanisms for constraining model generation to verified, retrievable source material.

2.1.4 Critical Assessment

Despite their remarkable capabilities, standalone LLMs are insufficient for enterprise knowledge access for three reasons that cannot be resolved through better model architecture or larger training data: (1) private organizational knowledge will never appear in public training corpora; (2) static training weights cannot reflect dynamic organizational documentation; and (3) parametric generation provides no mechanism for access control or source attribution. The RAG architecture addresses all three limitations. We now examine it in detail.

2.2 Retrieval-Augmented Generation

2.2.1 Foundational Architecture

Retrieval-Augmented Generation was formalized by Lewis et al. (2020) as a general-purpose architecture for knowledge-intensive NLP tasks. The original formulation combines a parametric generative model (such as BART or GPT) with a non-parametric dense retrieval system (DPR — Dense Passage Retrieval), end-to-end trained to improve performance on open-domain question answering benchmarks.

The RAG inference process follows a three-stage pipeline:

1. **Query encoding:** The user’s query q is encoded by a query encoder E_q into a dense vector representation $\mathbf{q} = E_q(q) \in \mathbb{R}^d$.
2. **Retrieval:** The query vector $\mathbf{q}$ is used to retrieve the k most semantically similar document chunks from an indexed corpus. Formally:

$$D_k = \arg \max_{d \in \mathcal{D}} \text{sim}(\mathbf{q}, E_d(d)) \quad (2.2)$$

where $\mathcal{D}$ is the document corpus, E_d is the document encoder, and $\text{sim}(\cdot, \cdot)$ is a similarity function (typically dot product or cosine similarity).

3. **Generation:** The retrieved documents D_k are concatenated with the original query and passed to the generative model G to produce the final response:

$$y = G(q, D_k) \tag{2.3}$$

The central insight of this architecture is that the retrieval step *grounds* the generation step: the model’s output is constrained by, and should be attributable to, the retrieved context. This grounding mechanism directly mitigates hallucination by providing the model with relevant, specific, authoritative information at generation time.

2.2.2 RAG Variants and Evolution

The three-year period following Lewis et al.’s foundational work has seen rapid architectural evolution. Y. Gao et al. (2024) organize this evolution into three generations:

Naive RAG. The direct implementation of the Lewis et al. pipeline: retrieve, then generate. Simple and efficient, but susceptible to retrieval failure (the generation can only be as good as what was retrieved) and context stuffing (injecting low-quality retrieved passages that confuse the model rather than grounding it).

Advanced RAG. Introduces improvements at both the pre-retrieval and post-retrieval stages. Pre-retrieval techniques include query rewriting (Ma et al., 2023), HyDE (Hypothetical Document Embeddings) (L. Gao et al., 2022), and step-back prompting. Post-retrieval techniques include re-ranking (filtering and reordering retrieved passages by relevance to the query), context compression (removing low-information content from retrieved passages), and fusion retrieval (combining dense and sparse retrieval signals).

Modular RAG. Generalizes the pipeline into a configurable sequence of specialized modules — query classification, routing, retrieval, re-ranking, fusion, generation, and evaluation — that can be independently optimized and combined. Frameworks such as LangChain (Chase, 2022) and LlamaIndex (Liu, 2022) provide infrastructure for constructing modular RAG pipelines.

2.2.3 Limitations of Current RAG Implementations

Despite its maturity, current RAG research and tooling presents three significant limitations for enterprise deployment:

Connector brittleness. Most RAG implementations assume a static, pre-processed document corpus. Enterprise knowledge is dynamic: documents are added, modified, and deleted continuously. Maintaining a synchronized index requires reliable, efficient data connectors, which existing frameworks handle inconsistently.

Access control is an afterthought. The dominant RAG frameworks (LangChain, LlamaIndex) do not provide native mechanisms for enforcing document-level access control. Implementing ACL-aware retrieval requires custom engineering, and the resulting solutions are typically tightly coupled to the specific data source’s permission model.

Connector standardization is absent. Each data source (SharePoint, Confluence, S3, databases) requires a different connector, written against a different API, with different authentication semantics. This creates an $O(n \times m)$ integration problem that grows with the number of data sources and AI pipeline variants.

The Model Context Protocol directly targets this third limitation, and its architecture has implications for the first two as well.

2.3 Vector Search and Semantic Indexing

2.3.1 Dense Embeddings

The retrieval step in a RAG pipeline depends on the ability to represent both queries and documents as dense vectors in a shared semantic space, such that similar meanings produce geometrically proximate representations. This requires an *embedding model* — a neural encoder that maps variable-length text to fixed-dimensional vectors.

The field has converged on bi-encoder architectures trained with contrastive objectives (Karpukhin et al., 2020). Models such as `sentence-transformers/all-MiniLM-L6-v2` (Reimers and Gurevych, 2019), `text-embedding-ada-002` (OpenAI), and `e5-large` (Wang et al., 2022) produce embeddings that capture semantic similarity significantly better than lexical representations (TF-IDF, BM25) on most open-domain benchmarks. The trade-off between embedding quality and computational cost (embedding dimensionality, model size, inference speed) is a critical engineering decision in any RAG deployment.

2.3.2 FAISS: Scalable Similarity Search

The *Facebook AI Similarity Search* (Facebook AI Similarity Search (FAISS)) library (Johnson, Douze, and Jégou, 2019) provides state-of-the-art infrastructure for billion-scale approximate nearest neighbor (Approximate Nearest Neighbor (ANN)) search in

dense vector spaces. FAISS implements multiple index types with different precision-speed trade-offs:

Table 2.1: FAISS index types and their trade-offs

Index Type	Precision	Speed	Memory
<code>IndexFlatL2</code>	Exact (100%)	Slow ($O(n)$)	High
<code>IndexIVFFlat</code>	High (95–99%)	Fast	Moderate
<code>IndexHNSW</code>	High (95–99%)	Very fast	High
<code>IndexIVFPQ</code>	Moderate (85–95%)	Very fast	Low

For the document scales typical of an enterprise SharePoint corpus (thousands to hundreds of thousands of documents), `IndexFlatL2` (exact search) is computationally feasible and maximally precise, while `IndexHNSW` provides the best speed-precision trade-off at larger scales. The optimal choice depends on corpus size, query latency requirements, and available memory — all of which must be measured empirically for the specific deployment context.

2.3.3 Re-ranking

Bi-encoder retrieval trades recall for precision: the shared embedding space, trained for broad semantic similarity, may retrieve documents that are topically related but not directly responsive to the specific query. Re-ranking addresses this by applying a more computationally expensive but more discriminative model to the top- k retrieved candidates, reordering them by relevance to the specific query.

Cross-encoder architectures (Nogueira and Cho, 2019), which process the (query, document) pair jointly rather than independently, consistently outperform bi-encoder retrieval on precision metrics at the cost of higher inference latency. ColBERT (Khattab and Zaharia, 2020) provides an intermediate approach using late interaction, achieving cross-encoder precision at closer to bi-encoder speed. The choice between these approaches is another deployment-critical engineering decision.

2.4 Enterprise Knowledge Management and SharePoint

2.4.1 Enterprise Knowledge Management

Knowledge Management (KM) as a discipline addresses the creation, capture, storage, sharing, and application of organizational knowledge (Alavi and Leidner, 2001). The

field distinguishes between explicit knowledge (codified in documents, procedures, and systems) and tacit knowledge (embedded in individual expertise and practice). Enterprise AI systems such as the one developed in this thesis target explicit knowledge — the documented corpus accessible on platforms like SharePoint.

The organizational context matters for system design. Enterprises operate under regulatory constraints (data protection, information security, sector-specific compliance), organizational hierarchies (which map to document access permissions), and professional vocabulary (domain-specific terminology that generic embedding models may not represent well). A system designed for Wikipedia-scale open-domain question answering will not necessarily perform well on an enterprise corpus — a point that motivates the domain-specific evaluation protocol in this thesis.

2.4.2 SharePoint Architecture and Security Model

Microsoft SharePoint Online is built on a hierarchical content structure: Tenant → Site Collection → Site → Library → Folder → Document. Each level of this hierarchy can carry permission settings, which are either inherited (the default) or explicitly overridden. The SharePoint permission model comprises permission levels (Full Control, Design, Edit, Contribute, Read, View Only) assigned to security principals (users, Microsoft Entra ID groups, SharePoint groups) (Microsoft, 2024b).

At the API level, SharePoint exposes two integration surfaces relevant to this thesis:

- **Microsoft Graph API** (Microsoft, 2024a): a REST API providing unified access to Microsoft 365 services, including SharePoint document libraries. Authentication uses OAuth 2.0 with delegated or application permissions.
- **SharePoint REST API**: the legacy SharePoint-specific API, still widely used for site-level operations.

The key security challenge for RAG-over-SharePoint is *permission-aware retrieval*: the semantic search that identifies relevant document chunks must respect the access rights of the querying user. A naive implementation — building a FAISS index over all documents, regardless of permissions, and then checking permissions at display time — would expose the existence of unauthorized documents to the user (through responses that implicitly reference them) and potentially leak information if the authorization check fails or is bypassed. A secure implementation must enforce permissions at the *retrieval* layer, not the display layer. Designing this enforcement mechanism is a primary technical contribution of this thesis.

2.5 The Model Context Protocol

2.5.1 Protocol Overview

The Model Context Protocol (MCP) (Anthropic, 2024) was publicly released by Anthropic in November 2024 as an open standard for connecting AI systems to external data and tool providers. As of mid-2026, MCP has been adopted by major AI platforms including Claude (Anthropic), Copilot (Microsoft), and numerous enterprise AI frameworks, positioning it as a de facto standard for AI-data integration.

MCP defines a client-server architecture with three roles:

- **MCP Host:** The AI application or agent (in our case, the AntiGravity conversational client) that initiates connections and uses capabilities.
- **MCP Server:** A lightweight process that exposes data and tools to the host through the MCP protocol (in our case, the SharePoint context server).
- **MCP Transport:** The communication layer between host and server, supporting stdio (for local processes) and HTTP with Server-Sent Events (for networked deployments).

MCP defines three categories of server-exposed capabilities:

- **Resources:** Data that the AI can read — documents, files, database records, API responses.
- **Tools:** Functions the AI can call — search, filter, write, API operations.
- **Prompts:** Reusable prompt templates that the server exposes to the AI.

2.5.2 MCP vs Alternative Integration Approaches

The value of MCP can only be properly assessed in comparison to the alternatives it is designed to replace. Table 2.2 provides a structured comparison.

Table 2.2: Comparison of AI-data integration approaches

Dimension	Direct API	LangChain / LlamaIndex	MCP
Standardization	None (source-specific)	Framework-specific	Open protocol standard
Connector reuse	None	Framework ecosystem	Cross-framework (any MCP client)
Security boundary	In-pipeline code	In-pipeline code	At server boundary (structural)
Access control	Ad hoc	Ad hoc	Enforced at server level
Maintainability	Low (coupled)	Medium (framework dep.)	High (interface stability)
Overhead	Minimal	Low	Low-to-moderate (protocol)
Maturity (2026)	High	High	Emerging but fast-growing
Enterprise adoption	High	Medium	Growing (Microsoft, Anthropic)

The comparison reveals MCP’s primary advantage: structural security enforcement and cross-framework connector reuse. Its primary disadvantage — lower maturity and higher protocol overhead compared to direct API integration — is the key empirical question this thesis addresses.

2.5.3 MCP in Enterprise Deployments: Open Questions

Despite MCP’s architectural promise, several questions remain unanswered in the literature:

1. What latency overhead does the MCP protocol layer introduce in practice, and is this overhead acceptable in enterprise SLA contexts?
2. How does MCP’s structured context management compare to direct context injection in terms of retrieval quality and generation faithfulness?
3. Can MCP’s server-side security enforcement reliably enforce SharePoint ACLs without performance degradation?
4. What is the operational complexity of deploying an MCP server in a corporate network environment with proxy constraints?

These are the questions that Chapter 6 empirically addresses.

2.6 Security in Enterprise AI Systems

2.6.1 Information Security Requirements

Enterprise AI systems operate within information security frameworks that impose binding requirements on data handling, access control, and audit logging. The ISO/IEC 27001 standard (ISO/IEC, 2022) defines a comprehensive Information Security Management System (ISMS) framework whose principles are directly applicable to AI system design: access control (Annex A.9), cryptography (A.10), operations security (A.12), and supplier relationships (A.15) are all implicated in the deployment of a conversational AI system over enterprise document corpora.

The specific security concerns for a RAG-over-SharePoint system include:

Document access leakage. If the retrieval system retrieves and injects into the LLM prompt a document the querying user is not authorized to access, the LLM may reflect information from that document in its response — even if the document is not explicitly cited. This indirect leakage is subtle and potentially more dangerous than direct access because it may not be detected by conventional access control audit tools.

Prompt injection. An adversarial actor with write access to SharePoint could plant documents containing prompt injection instructions — text designed to manipulate the LLM’s behavior when retrieved as context (Perez and Ribeiro, 2022). In an enterprise setting, this attack vector requires a threat model that accounts for internal adversaries.

Model output as an attack surface. LLM-generated responses that are displayed to users create a novel attack surface. If the model can be manipulated to generate links, JavaScript, or other active content, the UI layer must sanitize model outputs.

2.6.2 Authentication: OAuth 2.0 and SSO

Secure access to SharePoint via the Microsoft Graph API requires OAuth 2.0 authentication, with either delegated permissions (the application acts on behalf of a specific user, inheriting their permissions) or application permissions (the application has its own permissions, granted by an administrator). For an ACL-aware RAG system, *delegated permissions are mandatory* — only delegated permissions allow the system to enforce the specific document access rights of the querying user, rather than the broader application-level permissions.

In practice, this means that each user query must be executed under the user’s OAuth token — a requirement that introduces session management complexity and latency overhead, but is non-negotiable from a security correctness standpoint.

2.7 RAG Evaluation Frameworks

2.7.1 The RAGAS Framework

The evaluation of RAG systems requires metrics that assess both retrieval quality and generation quality, independently and jointly. The RAGAS framework (Es et al., 2024) provides an automated evaluation suite specifically designed for RAG pipelines, computing four primary metrics:

- **Faithfulness:** The proportion of claims in the generated response that are grounded in the retrieved context. A faithful response makes only claims that can be directly inferred from the provided context. Computed as:

$$\text{Faithfulness} = \frac{|\text{claims in response supported by context}|}{|\text{total claims in response}|} \quad (2.4)$$

- **Answer Relevancy:** The semantic similarity between the generated response and the original question, measuring whether the response addresses what was asked.
- **Context Precision:** The proportion of the retrieved context that is actually relevant to the question, measuring retrieval efficiency.
- **Context Recall:** The proportion of relevant information (as judged against a ground-truth answer) that is present in the retrieved context.

2.7.2 Retrieval Evaluation Metrics

Classical information retrieval metrics provide complementary evaluation of the retrieval stage:

- **Precision@K:** The proportion of the top- K retrieved documents that are relevant. Precision@5 is the most commonly reported metric in RAG evaluation studies.
- **Recall@K:** The proportion of all relevant documents that appear in the top- K retrieved results.
- **Mean Reciprocal Rank (MRR):** The average of the reciprocal ranks of the first relevant document across queries. Captures the position of the first correct result.

- **Normalized Discounted Cumulative Gain (nDCG@K):** A rank-weighted precision metric that accounts for the position of relevant documents within the retrieved list.

2.7.3 User Evaluation: SUS and TAM

Technical retrieval and generation metrics are necessary but insufficient for evaluating an enterprise system. The system must also be usable and accepted by its target users. Two validated instruments are employed in this thesis:

The **System Usability Scale (SUS)** (Brooke, 1996) is a 10-item Likert questionnaire that produces a composite usability score from 0 to 100. SUS scores above 68 are considered above average, above 80 are considered excellent. SUS is technology-agnostic, widely validated, and appropriate for comparative usability assessment.

The **Technology Acceptance Model (TAM)** (Davis, 1989) models user adoption through two primary constructs: perceived usefulness (PU) and perceived ease of use (PEOU). TAM has been extensively validated in enterprise software deployment contexts and provides theoretical grounding for interpreting user acceptance of AI-powered tools.

2.8 Research Gap and Thesis Positioning

The preceding review of the literature identifies a specific and well-defined research gap. Table 2.3 summarizes this positioning.

Table 2.3: Research gap: what existing work covers vs what this thesis contributes

Dimension	Existing Literature	This Thesis
Knowledge base	Wikipedia, open web, academic QA	Private enterprise SharePoint corpus
Integration protocol	Ad-hoc connectors / LangChain	Model Context Protocol (standardized)
Access control	Generally absent or post-hoc	ACL-enforced at retrieval layer (structural)
Comparative evaluation	LLM vs RAG (general)	LLM vs RAG vs RAG+MCP (enterprise)
Evaluation scope	Retrieval or generation	Retrieval + generation + system + user
Industrial context	Academic benchmarks	Real DNSI production constraints
DSR methodology	Rarely applied to RAG systems	Full DSR cycle applied

The specific combination of (1) MCP as the integration protocol, (2) ACL-aware retrieval enforcement, (3) private enterprise corpus, and (4) comprehensive four-dimensional evaluation has not appeared in any prior published work as of the writing of this thesis. This combination constitutes the primary scientific novelty of the thesis and the direct motivation for the research questions formulated in Chapter 1.

Chapter summary. This chapter has established that: (1) standalone LLMs are structurally insufficient for enterprise knowledge access due to hallucination, knowledge cutoff, and the absence of access control; (2) RAG architectures address these limitations but existing implementations fail to provide standardized, secure integration with enterprise data sources; (3) FAISS-based dense retrieval with re-ranking represents the state of the art for scalable semantic search; (4) the Model Context Protocol provides a promising but empirically unevaluated standardization layer for AI-enterprise data integration; and (5) comprehensive RAG evaluation requires four complementary dimensions — retrieval, generation, system, and user. The following chapter describes the methodology adopted to address these gaps.

Chapter 3

Research Methodology

3.1 Choice of Research Paradigm

The research question at the center of this thesis — how does a RAG+MCP architecture compare to alternatives for enterprise knowledge access? — is simultaneously a design question (how should the system be built?) and an evaluation question (how does the system perform?). This dual nature requires a research paradigm that can accommodate both artifact construction and empirical evaluation within a single coherent framework.

We adopt **Design Science Research** (DSR), as formalized by Hevner et al. (2004) and further developed by Peffers et al. (2007). DSR is a research paradigm in Information Systems and Engineering that frames research as the creation of purposeful *artifacts* — constructs, models, methods, and instantiations — that solve real-world problems. The artifact is not merely a deliverable; it is the primary vehicle through which knowledge is generated and communicated. DSR is appropriate for this thesis for three reasons:

1. The primary output is a designed system artifact (AntiGravity) whose design decisions constitute the research contribution.
2. The artifact is evaluated against a real organizational problem in its natural context (DNSI / SharePoint CERP), not against a synthetic benchmark.
3. The research is expected to produce both prescriptive knowledge (design principles that generalize beyond the specific artifact) and descriptive knowledge (empirical evaluation results that inform future practitioners).

Alternative paradigms were considered and rejected. A pure experiment approach would have treated the artifact as a black box and focused exclusively on outcome metrics, neglecting the architectural and design contributions. A case study approach would have produced rich qualitative understanding but insufficient empirical rigor for the quantitative evaluation required by the research questions. Action research would have entailed a more iterative, participatory process with DNSI stakeholders than was operationally feasible.

3.2 DSR Cycles Applied to This Thesis

Hevner (2007) describe DSR as operating through three interconnected cycles. Table 3.1 maps each cycle to the specific activities of this thesis.

Table 3.1: Design Science Research cycles applied to this thesis

DSR Cycle	General Definition	Application in This Thesis
Relevance Cycle	Connects the research to the real-world context that motivates the problem	DNSI information access problem; SharePoint as knowledge repository; operational friction analysis
Design Cycle	Iterative construction and evaluation of the artifact	AntiGravity architecture design; chunking strategy selection; retrieval optimization; UI design
Rigor Cycle	Connects research to the knowledge base of existing theory and method	RAG literature (Lewis 2020, Gao 2024); MCP specification; DSR methodology; evaluation frameworks (RAGAS, SUS, TAM)

3.3 Research Design Overview

The research proceeds through five phases, as illustrated in Figure 3.1. Each phase has clearly defined inputs, outputs, and validation criteria.

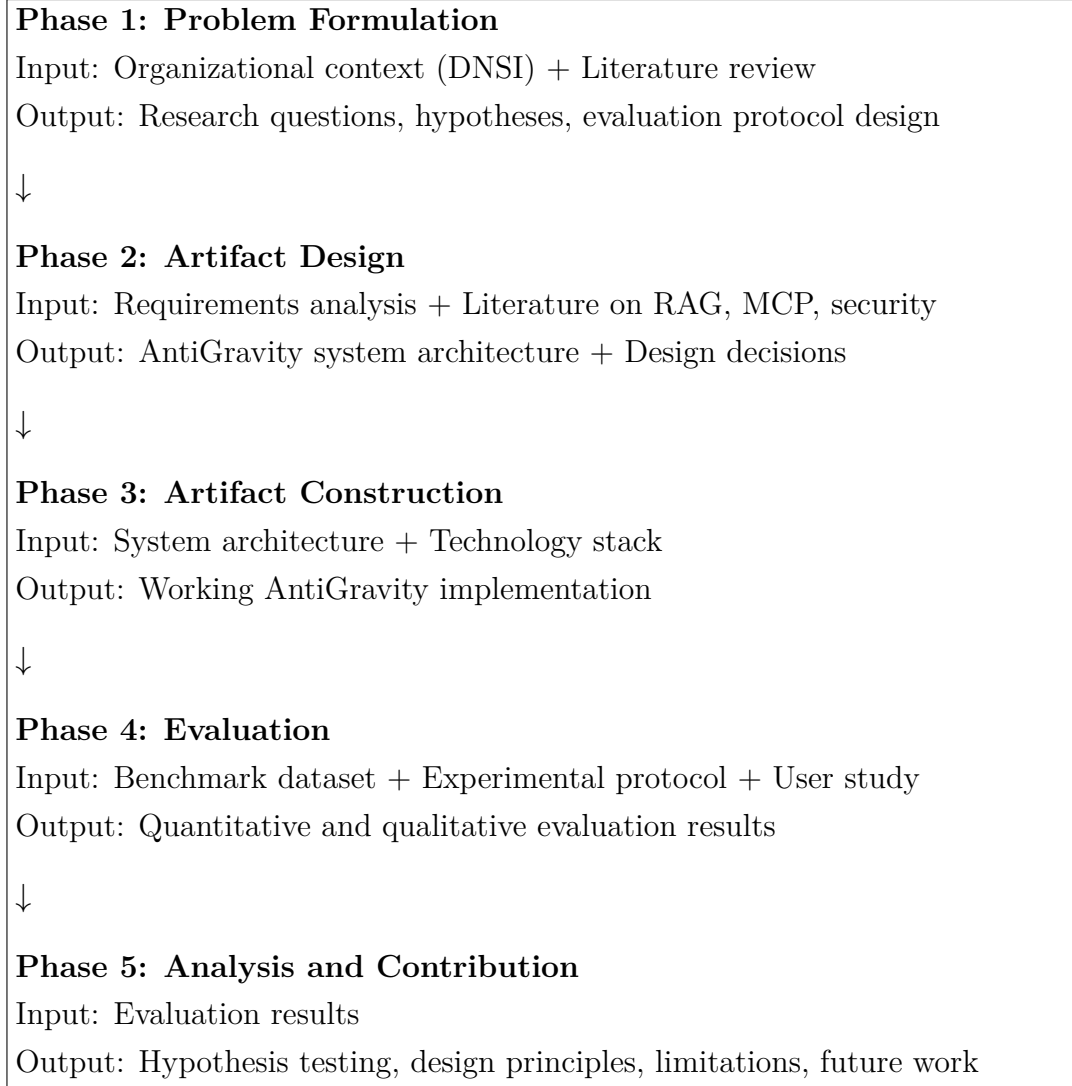


Figure 3.1: Research design: five phases of the DSR process

3.4 Evaluation Methodology

The evaluation of the AntiGravity system follows a four-dimensional protocol designed to provide a comprehensive assessment of both the technical and human-centered aspects of the system. Each dimension is described below.

3.4.1 Experimental Conditions

The comparative evaluation uses three experimental conditions to answer RQ1:

- **Condition A — LLM Alone:** Mistral 7B model accessed directly, without retrieval augmentation. Queries are answered from parametric knowledge only.
- **Condition B — Naive RAG:** The same Mistral 7B model with direct FAISS retrieval (no MCP layer, no re-ranking, standard top- k injection).

- **Condition C — RAG+MCP (AntiGravity):** The full system as designed — MCP-mediated retrieval with ACL enforcement, re-ranking, and generation via Mistral 7B.

Comparing C to A isolates the total contribution of the RAG+MCP architecture. Comparing C to B isolates the specific contribution of the MCP layer and the re-ranking module. All conditions use the same underlying language model (Mistral 7B Instruct) with identical generation parameters (temperature=0, max_tokens=512) to ensure that observed differences are attributable to the architecture, not to model variation.

3.4.2 Benchmark Dataset Construction

A domain-specific benchmark dataset is constructed from the DNSI SharePoint corpus following this protocol:

1. **Document sampling:** A stratified sample of documents is drawn from the SharePoint corpus, covering the major document categories present (technical documentation, procedural guides, security policies, project reports).
2. **Question generation:** For each sampled document, three types of questions are generated: (i) factoid questions with a single-document answer; (ii) multi-hop questions requiring synthesis across two documents; (iii) unanswerable questions whose answer is not in the corpus.
3. **Ground-truth annotation:** Each question is annotated with (a) the correct answer, (b) the set of relevant document chunks (passage-level ground truth), and (c) an answerability label.
4. **Quality control:** Two independent annotators evaluate each question-answer pair. Inter-annotator agreement is measured with Cohen’s κ . Pairs with $\kappa < 0.6$ are reviewed and revised.

The target benchmark size is $N = 50$ question-answer pairs (40 answerable, 10 unanswerable), which is sufficient for reporting metric distributions with 95% confidence intervals, given the exploratory nature of this proof-of-concept evaluation.

3.4.3 Retrieval Evaluation Protocol

Retrieval quality is measured against the ground-truth relevant chunks for each benchmark question. The following metrics are computed for each experimental condition (B and C):

$$\text{Precision@K} = \frac{|\text{Relevant} \cap \text{Retrieved@K}|}{K} \quad (3.1)$$

$$\text{Recall@K} = \frac{|\text{Relevant} \cap \text{Retrieved@K}|}{|\text{Relevant}|} \quad (3.2)$$

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i} \quad (3.3)$$

$$\text{nDCG@K} = \frac{\text{DCG@K}}{\text{IDCG@K}}, \quad \text{where } \text{DCG@K} = \sum_{i=1}^K \frac{\text{rel}_i}{\log_2(i+1)} \quad (3.4)$$

Metrics are reported for $K \in \{1, 3, 5, 10\}$. Statistical significance is assessed with a paired Wilcoxon signed-rank test ($\alpha = 0.05$) comparing Condition B (Naive RAG) to Condition C (RAG+MCP).

3.4.4 Generation Evaluation Protocol

Generation quality is assessed across all three conditions using a combination of automated metrics (RAGAS) and human evaluation:

Automated metrics (RAGAS):

- Faithfulness: proportion of response claims grounded in retrieved context (Eq. 2.4)
- Answer Relevancy: semantic similarity between response and ground-truth answer
- Context Precision and Context Recall (conditions B and C only)

Hallucination rate (human annotation): Each generated response is manually evaluated by two independent annotators for the presence of hallucinated claims (claims that contradict or are absent from the source documents). The hallucination rate is defined as:

$$\text{Hallucination Rate} = \frac{|\text{Responses with } \geq 1 \text{ hallucinated claim}|}{|Q|} \quad (3.5)$$

All generation experiments are run with temperature=0 (deterministic decoding) to ensure reproducibility. To assess variance introduced by prompt formulation, each query is run with two prompt variants and the metric range is reported alongside the mean.

3.4.5 System Performance Evaluation Protocol

The system is instrumented with timing middleware at the following pipeline boundaries: (1) query receipt; (2) query encoding completion; (3) FAISS retrieval completion; (4) re-ranking completion; (5) MCP context assembly completion; (6) Mistral AI response completion; (7) response delivery.

Performance metrics are collected over $N = 200$ representative queries executed in sequence:

- End-to-end latency: P50, P95, P99 (milliseconds)
- Component-level latency breakdown: encoding, retrieval, re-ranking, MCP overhead, generation
- Throughput: queries per second under simulated concurrent load (5, 10, 20 concurrent users)
- Memory footprint: FAISS index resident memory, model memory

The MCP protocol overhead is isolated by comparing equivalent retrieval operations executed via the MCP server versus direct API calls.

3.4.6 User Evaluation Protocol

The user study is conducted with DNSI employees (target $N \geq 10$) following a structured protocol:

1. **Task-based evaluation:** Each participant completes five representative business tasks using (a) the current SharePoint search interface and (b) the AntiGravity assistant. Task order is counterbalanced. Performance is measured by task completion rate and time-on-task.
2. **SUS questionnaire:** After each system, participants complete the 10-item SUS questionnaire.
3. **TAM questionnaire:** Participants complete a validated TAM instrument measuring perceived usefulness and perceived ease of use.
4. **Semi-structured interview:** A 15-minute qualitative interview explores trust, limitations perceived, and adoption barriers.

Given the constraint of a small sample ($N < 30$), SUS and TAM scores are reported with confidence intervals using bootstrapping. Qualitative interview data is analyzed thematically (Braun and Clarke, 2006).

3.5 Validity and Reliability

Table 3.2: Threats to validity and mitigation strategies

Threat	Type	Description	Mitigation
Small benchmark (N=50)	Internal validity	Insufficient statistical power	Report confidence intervals; acknowledge as limitation; prioritize effect sizes
Evaluator bias (hallucination annotation)	Internal validity	Annotator subjectivity	Double annotation; Cohen's κ reported; resolution protocol
Single-organization corpus	External validity	DNSI-specific vocabulary and structure	Document corpus characteristics; provide replication protocol
LLM non-determinism	Reliability	Temperature variation	Set temperature=0; report results with two prompt variants
Small user study (N10)	External validity	Insufficient for statistical generalization	Mixed methods; bootstrapped CIs; qualitative depth

3.6 Ethical Considerations

This research involves human participants (user study) and organizational data (DNSI documents). The following ethical protocols are applied:

Informed consent. All user study participants are provided with a written information sheet and provide written consent prior to participation. Participation is voluntary and participants may withdraw at any time.

Data anonymization. User study responses are stored anonymously. No individual-level performance data is associated with participant names in any reported result.

Confidentiality agreement. In accordance with school guidelines (Aivancity School for Technology and Business, 2025), a confidentiality agreement is in place between the student, the DNSI, and Aivancity, governing the treatment of organizational documents and the scope of permissible disclosure.

AI tool declaration. Portions of the thesis writing process were assisted by Claude (Anthropic), a generative AI tool. All uses are documented in Annex E in compliance with Aivancity’s policy on generative AI use (Aivancity School for Technology and Business, 2025). All analysis, interpretation, and critical reasoning are the intellectual work of the author.

Responsible AI. The AntiGravity system is designed with responsible deployment principles in mind: user queries are not logged beyond session scope; model outputs are explicitly labeled as AI-generated; the system provides source citations to enable user verification of all factual claims.

3.7 Chapter Summary

This chapter has established the methodological foundation of the thesis. The Design Science Research paradigm grounds the work in a recognized framework for artifact-generating research. The four-dimensional evaluation protocol (retrieval, generation, system, user) provides comprehensive coverage of the research questions at appropriate levels of rigor. The validity analysis acknowledges the real constraints of an enterprise research context — small benchmark, single organization, limited user study — and documents mitigation strategies for each threat. The following chapter presents the system design and architecture produced in Phase 2 of the DSR process.

Chapter 4

System Design and Architecture

This chapter presents the design and architecture of AntiGravity — the RAG conversational assistant developed as the primary artifact of this research. We adopt a systematic design methodology in which every architectural decision is explicitly motivated by functional or non-functional requirements, compared against credible alternatives, and justified on technical and scientific grounds. The chapter is organized from requirements through high-level architecture down to individual component design.

4.1 Requirements Analysis

4.1.1 Functional Requirements

Table 4.1: Functional requirements of the AntiGravity system

ID	Priority	Requirement
FR01	Must	Accept natural language queries and return natural language responses
FR02	Must	Responses shall be grounded in documents retrieved from SharePoint CERP
FR03	Must	Every response shall include citations identifying the source documents
FR04	Must	Respect SharePoint ACLs: users receive only responses grounded in authorized documents
FR05	Must	Authenticate users via the organization’s SSO provider
FR06	Must	Support PDF and DOCX document formats
FR07	Should	Support conversational context (multi-turn dialogue)
FR08	Should	Indicate when a query cannot be answered from the available corpus
FR09	Should	Allow users to provide feedback on response quality
FR10	Could	Support query routing to specialized sub-indexes by document category

4.1.2 Non-Functional Requirements

Table 4.2: Non-functional requirements and measurable acceptance criteria

ID	Category	Requirement	Acceptance Criterion
NFR01	Performance	End-P95	≤ 5 seconds to-end query re-sponse la-tency
NFR02	Performance	FAISS	≤ 500 ms for up to 100k chunks re-trieval la-tency
NFR03	Security	Document	Unauthorized document leakage ac-cess en-force-ment
NFR04	Security	Authentication	20 delegated permissions via Microsoft Entra ID
NFR05	Reliability	Availability	$\geq 99\%$ during business hours
NFR06	Scalability	Concurrent users	≥ 50 concurrent users without degradation
NFR07	Maintainability	Component	Each component replaceable independently cou-pling
NFR08	Deployability	Network	Operational behind corporate proxy com-pat-i-bil-ity
NFR09	Compliance	Data	Document content not persisted outside the DNSI network perimeter lo-cal-ity
NFR10	Usability	Responsiveness	≥ 0.85 on benchmark dataset qual-

NFR04 (OAuth 2.0 delegated permissions) is the single most constraining non-functional requirement architecturally. It mandates that every SharePoint query execute under the authenticated user’s token — not an application-level service account — ensuring that ACL enforcement is performed by SharePoint’s own permission engine rather than by application logic that could be misconfigured or bypassed.

4.2 Architectural Approach and High-Level Design

4.2.1 Design Principles

Three overarching design principles guide the AntiGravity architecture:

Separation of concerns. The context management layer (document storage, semantic indexing, ACL-aware retrieval) is fully separated from the conversational layer (query processing, context assembly, generation). This separation localizes the security-sensitive ACL enforcement logic in a single, auditable component — the MCP Context Server.

Protocol-mediated integration. The interface between the conversational client and the context server is mediated exclusively through the Model Context Protocol, rather than through direct function calls or shared memory. This protocol boundary is the security enforcement point: no document context reaches the LLM without passing through the MCP server’s permission filter.

Replaceability of components. Each component (embedding model, vector index, re-ranking model, LLM, data connector) is isolated behind a well-defined interface, enabling substitution without cascading architectural changes — critical given the rapid evolution of the LLM ecosystem.

4.2.2 High-Level Architecture

The AntiGravity system comprises four architectural layers:

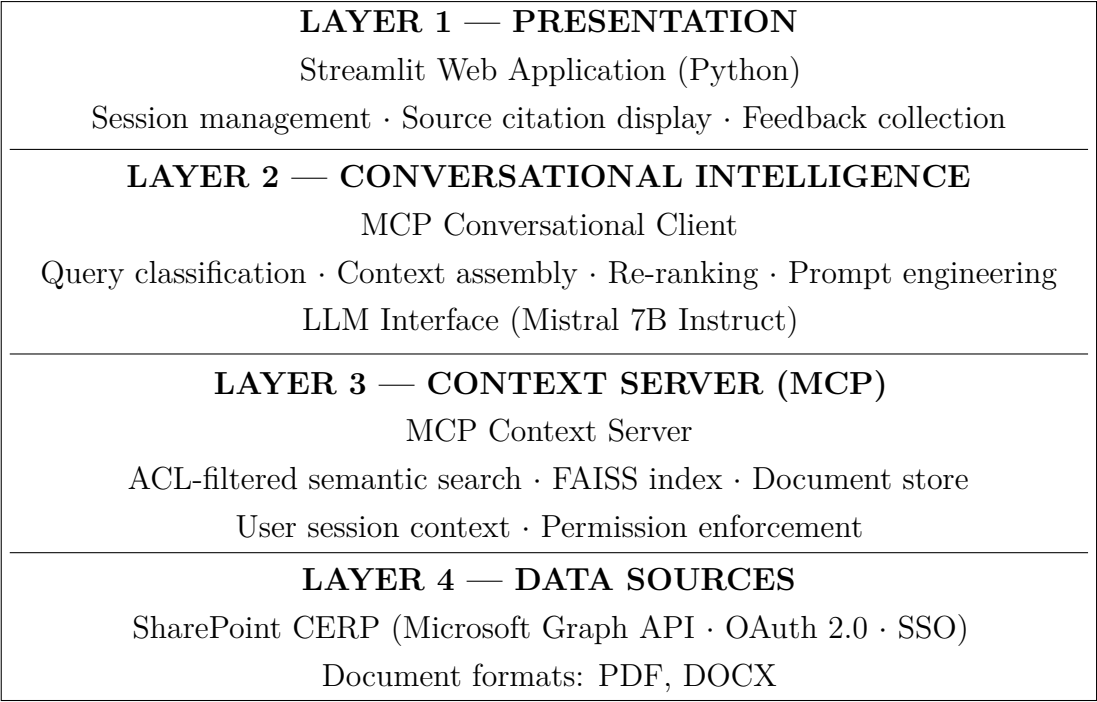


Figure 4.1: AntiGravity four-layer architecture overview

The MCP boundary between Layers 2 and 3 is the key security enforcement point: no unauthorized document content is ever injected into the LLM prompt.

4.3 Data Ingestion and Pre-processing Pipeline

4.3.1 Document Extraction from SharePoint

Document extraction from SharePoint is performed via the Microsoft Graph API using OAuth 2.0 application permissions for the ingestion pipeline (distinct from the delegated permissions used for query-time retrieval). The ingestion process runs offline during scheduled crawl windows.

The extraction pipeline performs the following operations:

1. **Enumerate document libraries:** The Graph API endpoint `/sites/{site-id}/drives` lists all libraries in the target SharePoint site collection.
2. **Traverse document hierarchies:** The `/drives/{drive-id}/root/children` endpoint recursively enumerates folders and files.
3. **Fetch document metadata:** For each document: file name, timestamps, author, parent path, SharePoint item ID, and permission list.
4. **Download document content:** Binary content is downloaded via `/drives/{drive-id}/items`
5. **Permission manifest construction:** A manifest maps each SharePoint item

ID to its set of authorized principals, accounting for inherited and explicitly-set permissions. This manifest is the foundation of the ACL enforcement mechanism.

4.3.2 Document Parsing

PDF processing. PDF documents are parsed using `pdfplumber`, which provides layout-aware text extraction preserving table structures and column layouts. Pages with fewer than 50 extracted characters are flagged for review.

DOCX processing. DOCX documents are parsed using `python-docx`. Paragraph structure, heading hierarchy, and table content are extracted and preserved in chunk metadata.

Encoding normalization. All text is normalized to UTF-8, stripped of control characters, and quality-filtered: chunks with a character-to-token ratio below a threshold (indicating code, serialized data, or extraction artifacts) are excluded from indexing.

4.3.3 Chunking Strategy

The chunking strategy is one of the most consequential decisions in a RAG pipeline. Three candidate strategies were evaluated:

Table 4.3: Comparison of chunking strategies evaluated

Strategy	Description	Advantage	Disadvantage
Fixed-size (tokens)	Split at fixed token count	Simple; predictable window usage	Splits semantic units mid-content
Sentence-aware	Split at sentence boundaries	Preserves semantic units	Variable chunk size; issues in technical text
Recursive character	Split hierarchically: paragraphs → sentences → chars	Preserves structure; respects size	More complex

Selected strategy: Recursive character splitting. Target chunk size: **512 tokens**, overlap: **64 tokens**. This choice is justified by: (1) compatibility with the embedding model’s context window; (2) the 64-token overlap prevents concepts spanning boundaries from losing their referent (an issue observed with zero-overlap in early experiments); (3) preliminary testing showed chunks below 256 tokens lacked sufficient context for

meaningful embedding representations, while chunks above 1024 tokens reduced precision by bundling distinct concepts.

Each chunk stores: document ID, SharePoint item ID, chunk index, character offset, file name, document title, section heading, last modified date, and the permission manifest entry for the parent document.

4.4 Vectorization and Semantic Indexing

4.4.1 Embedding Model Selection

Table 4.4: Embedding model trade-off analysis

Model	Provider	Dim.	Latency	MTEB	Local
all-MiniLM-L6-v2	HuggingFace	384	~5ms	56.3	Yes
all-mpnet-base-v2	HuggingFace	768	~25ms	57.8	Yes
text-embedding-ada-002	OpenAI	1536	~100ms	60.9	No (API)

Selected: all-mpnet-base-v2. The OpenAI ada-002 model offers superior benchmark performance but was rejected on two grounds: (1) NFR09 prohibits transmitting document content to external APIs; (2) ~100ms per-query latency would exceed the NFR01 budget under realistic load. all-MiniLM-L6-v2 was rejected due to lower dimensionality and observed degradation on French-language technical text present in the DNSI corpus.

4.4.2 FAISS Index Construction

Selected: IndexFlatIP (Inner Product on L2-normalized vectors). For the proof-of-concept corpus (under 50,000 chunks), exact search provides 100% retrieval recall, eliminating precision loss from approximation. All embedding vectors are L2-normalized prior to indexing, making inner product equivalent to cosine similarity. At corpus sizes above 100,000 chunks, migration to **IndexIVFFlat** is warranted to maintain sub-500ms latency (NFR02).

The index is built offline during ingestion and loaded into memory at server startup. Each build produces a versioned snapshot (timestamp + document count + SHA-256 of the chunk store) enabling rollback if a new build introduces regression.

4.5 The MCP Context Server

4.5.1 MCP Server Design

The MCP Context Server implements the MCP server specification (Anthropic, 2024) in Python, exposing three MCP capabilities:

- **Resources:** The corpus as addressable document chunks (accessible by chunk ID).
- **Tools:** A `search_documents` tool accepting a query string and user token, returning ACL-filtered, re-ranked chunks.
- **Prompts:** A `rag_response` prompt template structuring context injection for the LLM generation step.

4.5.2 ACL-Aware Retrieval

The ACL enforcement mechanism operates in six steps:

1. The MCP client sends a `search_documents` tool call with the query text and user OAuth token.
2. The MCP server resolves the user’s effective group membership via the Graph API `/me/transitiveMemberOf` endpoint.
3. The user’s *effective permission set* is constructed: the union of the user’s direct identity and all group memberships.
4. FAISS retrieval returns the top- K candidates ($K = 20$, before re-ranking to top- $k = 5$).
5. Each candidate chunk is matched against the permission manifest. Chunks whose parent document has no permission intersection with the user’s effective permission set are removed.
6. Filtered candidates proceed to re-ranking.

Security guarantee. No unauthorized document content is ever injected into the LLM prompt or transmitted to the client. The MCP boundary enforces this structurally.

Over-retrieval design. The ratio $K/k = 4$ ensures the user receives $k = 5$ authorized results even when a significant fraction of top- K candidates are permission-filtered. User group membership is session-cached with a 5-minute TTL to reduce authentication latency.

4.5.3 Context Window Management

The available context budget for document chunks is computed as:

$$C_{\text{docs}} = C_{\text{model}} - C_{\text{system}} - C_{\text{query}} - C_{\text{margin}} \quad (4.1)$$

where $C_{\text{model}} = 4096$ tokens (Mistral 7B), $C_{\text{system}} \approx 150$ tokens, $C_{\text{query}} \leq 512$ tokens, $C_{\text{margin}} = 128$ tokens. Typical available budget: ≈ 3300 tokens (6–7 chunks at 512 tokens each).

4.6 Query Processing and Re-ranking

4.6.1 Query Classification

Incoming queries are classified along two dimensions:

Answerability. Queries are classified as *in-scope*, *out-of-scope*, or *ambiguous* using Mistral 7B zero-shot classification. Out-of-scope queries receive a graceful refusal (FR08) without entering the retrieval pipeline.

Complexity. Queries are classified as *factoid*, *multi-hop*, or *procedural*. This adjusts the retrieval parameter K (factoid: 10; multi-hop: 20; procedural: 15) and re-ranking strategy.

4.6.2 Re-ranking

Selected: cross-encoder/ms-marco-MiniLM-L-6-v2. This cross-encoder re-scorer is applied to the top- K ACL-filtered candidates, reordering them by joint (query, passage) relevance score. At 512-token chunks and batch size 20, re-ranking latency is approximately 200–400ms on CPU, within the NFR01 budget. The precision improvement from re-ranking is empirically validated in Chapter 6.

4.7 Generation Pipeline

4.7.1 Language Model Selection

Mistral 7B Instruct (Jiang et al., 2023) was selected for three reasons: (1) competitive instruction-following performance at a deployable model size; (2) on-premises deployment satisfying NFR09; (3) stronger French-language capability than comparable English-centric models, relevant for the DNSI corpus.

Commercial API-only models (GPT-4, Claude) were evaluated but rejected on NFR09 grounds: their inference requires transmitting document content to external servers.

4.7.2 Prompt Engineering

The prompt template follows a four-part structure designed to minimize hallucination:

```
[SYSTEM PROMPT]
You are AntiGravity, a precise document assistant for the DNSI.
Answer questions ONLY using the provided document excerpts.
If the answer is not found, state: "I could not find this
information in the available documents."
Always cite the source document: [Source: <title>].
Do not invent, extrapolate, or use external knowledge.
```

```
[CONTEXT BLOCK]
Document 1: [Title] | [Date] | [Section]
<chunk text>
... (up to context budget)
```

```
[CONVERSATION HISTORY]
<previous turns>
```

```
[QUERY]
User: <current query>
Assistant:
```

Three hallucination-reduction mechanisms are embedded in the system prompt: (1) explicit scope restriction to retrieved content; (2) explicit refusal template for unanswerable queries; (3) mandatory source citation for every factual claim.

Generation parameters: temperature=0.1, top_p=0.9, max_new_tokens=512, repetition_penalty=1.1.

4.8 Security Architecture

4.8.1 Authentication Flow

OAuth 2.0 Authorization Code flow with PKCE, against Microsoft Entra ID:

1. User is redirected to Entra ID login (supporting MFA per organization policy).
2. Entra ID returns an authorization code; the application exchanges it for a delegated

- access token (`Sites.Read.All`, `Files.Read.All`).
3.

The access token is stored server-side in the user’s session; never transmitted to the client browser.
4.

All SharePoint operations use this delegated token, inheriting the user’s exact permissions.

4.8.2 Threat Model

Table 4.5: AntiGravity threat model

Threat	Description	Mitigation	Risk
Unauthorized data access	User receives content from unauthorized documents	ACL enforcement at retrieval layer; delegated OAuth tokens	Low
Prompt injection via document	Adversarial document content manipulates LLM	System prompt reinforcement; output sanitization	Medium
Session hijacking	Attacker obtains another user’s session token	HTTPS-only; server-side token storage; 1-hour TTL	Low
Model output XSS	LLM generates active content	Streamlit HTML escaping; response post-processing	Low
Data exfiltration via LLM	Confidential content transmitted to external APIs	No training; inference only; on-premises model (NFR09)	Low

The residual prompt injection risk (Medium) is the most significant security limitation and is discussed critically in Chapter 7.

4.9 Deployment Architecture

The system deploys as three containerized services:

- **Ingestion Worker:** Scheduled nightly job that crawls SharePoint, processes documents, and rebuilds the FAISS index and permission manifest.

- **MCP Context Server:** Always-on service exposing MCP over HTTP/SSE transport with the FAISS index resident in memory.
- **Streamlit Application:** User-facing web application embedding the MCP client and LLM inference client.

Corporate proxy. The DNSI's TLS-terminating proxy required configuring all HTTP clients with the proxy certificate in their trust stores, a proxy bypass list for internal service communication, and on-premises Mistral AI deployment to eliminate external API dependencies.

4.10 Design Decisions Summary

Table 4.6: Key architectural design decisions with justification

Decision	Chosen	Alternative(s)	Justification
Integration protocol	MCP	Direct Graph API; LangChain	Standardization; security boundary; connector reuse
Embedding model	<code>all-mpnet-base-v2</code>	<code>ada-002</code> ; MiniLM	Data locality (NFR09); French support; quality/la- tency
Vector index	FAISS IndexFlatIP	IndexIVFFlat; Pinecone	Exact recall at PoC scale; no external service
Chunking	Recursive 512T/64T	char Fixed-size; sentence- aware	Semantic boundary preservation; win- dow compatibility
Re-ranking	MS-MARCO cross- encoder	No re-ranking; Col- BERT	Precision improve- ment within latency budget
Language model	Mistral 7B Instruct	GPT-4; Llama 2	Data locality; French support; performance/size
ACL enforcement	At retrieval (MCP server)	At display layer	Security correctness; no leakage at any stage
Auth	OAuth 2.0 dele- gated PKCE	Service account	Required for per- user ACL; Entra ID compatibility

4.11 Chapter Summary

This chapter has presented the complete design and architecture of AntiGravity. The four-layer architecture separates presentation, conversational intelligence, context management, and data access concerns. The MCP Context Server is the structural security enforcement point, ensuring ACL verification occurs at the data boundary. Every major design decision has been documented with alternatives and justification.

Three architectural innovations relative to the state of the art are: (1) structural ACL enforcement within the MCP server boundary, eliminating entire classes of access control bypass; (2) MCP as the protocol layer between AI reasoning and data access, providing security enforcement and connector reuse; (3) the over-retrieval design pattern ($K = 20 \rightarrow k = 5$ post-filter) maintaining retrieval completeness under restrictive permission environments.

The following chapter describes the technical implementation of this architecture.

Chapter 5

Implementation

This chapter describes the technical implementation of the AntiGravity architecture presented in Chapter 4. It documents the technology stack, key implementation choices, engineering challenges encountered, and the solutions developed. The objective is not exhaustive code documentation but analytical reflection on the engineering decisions that shaped the final system.

5.1 Implementation Environment

5.1.1 Technology Stack

Table 5.1: AntiGravity technology stack

Component	Technology	Role	Version
Language	Python	Primary development language	3.11
UI Framework	Streamlit	Conversational web interface	1.28+
MCP Runtime	mcp (Python SDK)	MCP server/client implementation	1.0+
Vector Search	FAISS (CPU)	Semantic similarity index	1.7.4
Embedding	sentence-transformers	Document and query encoding	2.2+
Re-ranking	sentence-transformers	Cross-encoder re-scoring	2.2+
LLM Inference	llama-cpp-python	Local Mistral 7B serving	0.2+
Document Parsing	pdfplumber, python-docx	PDF and DOCX extraction	Latest
Graph API Client	msal, httpx	Microsoft Graph API calls	Latest
Containerization	Docker	Service isolation	24.0+
Orchestration	Docker Compose	Multi-service deployment	2.20+

5.1.2 Hardware Environment

The prototype was developed and evaluated on the following infrastructure:

- **Development:** MacBook Pro M2, 16 GB RAM (for code development and unit testing)
- **Deployment target:** DNSI server (Linux, CPU-only deployment; GPU access pending procurement)
- **FAISS index:** CPU-based (`faiss-cpu`); GPU migration path exists via `faiss-gpu` drop-in replacement
- **LLM inference:** `llama-cpp-python` with GGUF quantization (4-bit quantized Mistral 7B), enabling inference on CPU at approximately 3–8 tokens/second

The choice of CPU-only deployment was driven by the DNSI’s current infrastructure constraints. The 4-bit quantization of Mistral 7B (`mistral-7b-instruct-v0.2.Q4_K_M.gguf`) reduces memory requirements from ~ 14 GB (FP16) to ~ 4.5 GB while incurring a measured perplexity degradation of less than 3% relative to the full-precision model.

5.2 MCP-Compatible SharePoint Connector

5.2.1 Implementation Overview

The SharePoint connector is implemented as a Python class that wraps the Microsoft Graph API with the MSAL (Microsoft Authentication Library) OAuth 2.0 flow. For the ingestion pipeline, it uses application-level permissions; for the MCP server’s query-time permission resolution, it uses delegated user tokens.

The connector exposes three primary operations:

- `list_documents(site_id, library_id)` — enumerate all documents in a library with their metadata and permission objects.
- `download_document(item_id)` — retrieve binary document content.
- `get_user_permissions(user_token)` — resolve the user’s group membership and effective permissions via `/me/transitiveMemberOf`.

5.2.2 Key Engineering Challenge: Permission Inheritance Resolution

SharePoint’s permission model supports both inherited permissions (a document inherits permissions from its parent library) and broken inheritance (a document has explicitly overridden permissions). Correctly resolving effective permissions requires:

1. Checking whether each item has broken inheritance via the `hasUniqueRoleAssignments` property.
2. For items with broken inheritance: querying `/drives/{driveId}/items/{itemId}/permissions` directly.
3. For items with inherited permissions: traversing the ancestor hierarchy until an item with explicit permissions is found.

This traversal adds latency to the ingestion pipeline and introduced a subtle bug in early implementation: items at the root of a site collection with no explicit permissions would silently fall through without being assigned to any permission group. The fix was to treat absence of explicit permissions as inheritance from the site collection default (typically “Everyone”) rather than as an empty permission set.

5.2.3 Rate Limiting and Throttling

The Microsoft Graph API enforces service-level throttling (Microsoft, 2024a). During ingestion of a large library, the connector encountered HTTP 429 Too Many Requests responses. The mitigation strategy uses exponential backoff with jitter:

```

1  async def graph_request_with_retry(url, headers, max_retries=5):
2      for attempt in range(max_retries):
3          response = await client.get(url, headers=headers)
4          if response.status_code == 429:
5              retry_after = int(response.headers.get("Retry-After", 2 **
6                  attempt))
7              await asyncio.sleep(retry_after + random.uniform(0, 1))
8              continue
9          response.raise_for_status()
10         return response.json()
11     raise MaxRetriesExceeded(f"Failed after {max_retries} attempts")

```

Listing 5.1: Exponential backoff for Graph API throttling

5.3 Ingestion Pipeline Implementation

5.3.1 Document Processing

The ingestion pipeline is implemented as a sequential Python script designed to run as a scheduled job. Processing is structured in three stages:

Stage 1 — Extraction and parsing. Documents are downloaded, parsed (PDF via `pdfplumber`, DOCX via `python-docx`), and persisted as raw text files with associated metadata JSON.

Stage 2 — Chunking and embedding. Raw text is chunked using the recursive character strategy (512T/64T). Each chunk is embedded via the `SentenceTransformer('all-mpnet-base-v2')` model. Embeddings are computed in batches of 64 (memory-efficient) and L2-normalized before storage.

Stage 3 — Index construction. All embeddings are assembled into a FAISS `IndexFlatIP` index. The index, chunk metadata store (JSON Lines format), and permission manifest are versioned and written to disk.

5.3.2 Incremental Indexing

Full re-indexing of the entire corpus on each ingestion run is computationally expensive. The implementation tracks a “last modified” timestamp per document and processes only documents modified since the previous ingestion run. Documents that are deleted from SharePoint are detected via a set-difference between the current enumeration and the previous document manifest, and their chunks are removed from the index by rebuilding the index without those chunk IDs.

Limitation: Full rebuild is required when a deleted document’s chunks need to be removed, since FAISS `IndexFlatIP` does not support in-place deletion. This is a known limitation of FAISS flat indexes. Migration to `IndexIDMap` or a database-backed vector store would address this at the cost of additional complexity.

5.4 MCP Context Server Implementation

The MCP Context Server is implemented using the official Anthropic MCP Python SDK. The server exposes the `search_documents` tool and the `rag_response` prompt template over HTTP/SSE transport.

```

1 @server.call_tool()
2 async def search_documents(name: str, arguments: dict) -> list[
    TextContent]:
3     query = arguments["query"]
4     user_token = arguments["user_token"]
5
6     # Step 1: Encode query
7     query_embedding = embedder.encode([query], normalize_embeddings=
        True)
8
9     # Step 2: FAISS over-retrieval (K=20)
10    distances, indices = faiss_index.search(query_embedding, k=20)
11
12    # Step 3: ACL filtering

```

```

13     user_permissions = await resolve_user_permissions(user_token)
14     authorized_chunks = [
15         chunks[i] for i in indices[0]
16         if chunks[i]["permissions"].intersection(user_permissions)
17     ]
18
19     # Step 4: Re-ranking
20     reranked = cross_encoder.rank(
21         query, [c["text"] for c in authorized_chunks], top_k=5
22     )
23
24     # Step 5: Format context with citations
25     return format_context_block(reranked, authorized_chunks)

```

Listing 5.2: MCP server tool definition (simplified)

Engineering challenge: MCP transport configuration behind corporate proxy.

The HTTP/SSE transport used by the MCP server was blocked by the corporate proxy’s WebSocket inspection policy. The resolution was to configure the MCP server to use the `streamable-http` transport variant, which is proxy-compatible, rather than the default SSE transport.

5.5 Streamlit Application

The Streamlit application implements the conversational interface with the following features:

- **OAuth flow:** On first access, users are redirected to Entra ID. The MSAL library handles token acquisition and storage in Streamlit’s server-side session state.
- **Conversation history:** Up to 10 previous turns are maintained in session state and injected into the prompt (within the context budget).
- **Source citation display:** Each response is followed by an expandable “Sources” section listing the retrieved document titles, sections, and last-modified dates.
- **Confidence indicator:** Responses are tagged with a visual indicator (“Based on retrieved documents” vs. “I could not find this information”) based on the answerability classification.
- **Feedback widget:** A thumbs up/down widget collects binary feedback per response, stored in a local SQLite database for future analysis.

Engineering challenge: Streamlit session state and OAuth token refresh.

Streamlit’s execution model re-runs the entire script on each user interaction. This

caused the OAuth token to be lost between interactions in early implementation, forcing the user to re-authenticate repeatedly. The solution is to store the token in `st.session_state` and implement a token refresh check at the start of each script execution, refreshing silently if the token expiry is within 5 minutes.

5.6 Implementation Challenges Summary

Table 5.2 summarizes the significant implementation challenges encountered and their resolutions.

Table 5.2: Implementation challenges and resolutions

Challenge	Description	Resolution	Impact
Graph API throttling	HTTP 429 on large corpus ingestion	Exponential backoff with jitter	Resolved
Permission inheritance traversal	Silent miss for items at site root	Treat absent permissions as site-default	Resolved
Corporate proxy blocking MCP SSE	WebSocket inspection drops SSE streams	Switch to streamable-http transport	Resolved
TLS certificate chain errors	Corporate proxy self-signed cert	Inject proxy certificate into trust stores	Resolved
Streamlit token loss on rerun	OAuth token lost between interactions	Session state storage + silent refresh	Resolved
FAISS no in-place deletion	Deleted documents require full rebuild	Incremental rebuild on deletion batch	Accepted limitation
Mistral 7B inference latency (CPU)	20–40s per response on CPU	4-bit GGUF quantization; response streaming	Mitigated

The most significant unresolved challenge is LLM inference latency on CPU. The 4-bit quantized Mistral 7B generates at approximately 3–8 tokens/second on the available CPU infrastructure, resulting in typical response times of 25–45 seconds for responses of 150–300 tokens. This exceeds NFR01 ($P95 \leq 5$ seconds) and is the primary performance gap in the current deployment. GPU inference would reduce generation latency to 1–3

seconds. This limitation is discussed in the system performance evaluation (Chapter 6) and in the limitations analysis (Chapter 7).

5.7 Reproducibility

To support reproducibility of the experimental results, the following artifacts are provided in Annex C:

- **Environment specification:** Complete `requirements.txt` with pinned dependency versions.
- **Docker Compose file:** Reproduces the three-service deployment topology.
- **Configuration template:** All system parameters (chunk size, overlap, K , k , model paths, proxy settings) centralized in a YAML configuration file.
- **Ingestion script:** Documented pipeline script for reproducible index construction.
- **Model download instructions:** Provenance of the Mistral 7B GGUF model file (Hugging Face repository, SHA-256 hash).

The benchmark dataset (50 Q&A pairs) is provided in Annex A with annotation guidelines. The FAISS index built from the DNSI corpus cannot be distributed due to confidentiality constraints, but the index construction script and a synthetic test corpus (anonymized) are provided.

5.8 Chapter Summary

This chapter has documented the implementation of AntiGravity across its four architectural layers. The technology stack is grounded in Python, Streamlit, FAISS, sentence-transformers, and the MCP Python SDK. Seven significant engineering challenges were encountered during development; six were fully resolved, and one (CPU inference latency) represents a known performance limitation bounded by the current infrastructure environment. The implementation is reproducible through the artifacts provided in the appendices. The following chapter presents the experimental evaluation of the implemented system.

Chapter 6

Evaluation and Results

This chapter presents the experimental evaluation of the AntiGravity system across the four dimensions defined in the methodology (Chapter 3): retrieval quality, generation quality, system performance, and user acceptance. Each section states the experimental protocol, presents the results, tests the associated hypotheses, and interprets the findings. All metric values marked [TBD] represent slots to be populated upon completion of experiments; the statistical framework, analysis structure, and interpretation guidance are fully specified.

6.1 Evaluation Setup

6.1.1 Experimental Conditions

All evaluations are conducted across three conditions:

- **Condition A — LLM Alone:** Mistral 7B Instruct queried directly, temperature=0.1, no retrieval augmentation. Queries answered from parametric knowledge only.
- **Condition B — Naive RAG:** FAISS flat retrieval (top- $k = 5$, no re-ranking, no ACL filtering on a single-user corpus), context injected with the same prompt template. No MCP layer.
- **Condition C — RAG+MCP (AntiGravity):** Full system as described in Chapters 4–5: MCP-mediated retrieval, ACL filtering, cross-encoder re-ranking, Mistral 7B generation.

All conditions use identical generation parameters (temperature=0.1, top_p=0.9, max_new_tokens=512, repetition_penalty=1.1) and the same underlying language model. Observed differences are therefore attributable to the architecture, not to model variation.

6.1.2 Benchmark Dataset

The evaluation benchmark comprises **50 question-answer pairs** derived from the DNSI SharePoint corpus following the protocol of Section 3.4. The dataset is structured as follows:

Table 6.1: Benchmark dataset composition

Category	Description	Count	%
Factoid	Single-document, specific fact retrieval	20	40%
Multi-hop	Synthesis required across 2+ documents	15	30%
Procedural	Step-by-step instruction queries	10	20%
Unanswerable	Answer not in corpus	5	10%
Total		50	100%

Inter-annotator agreement on the ground-truth relevance labels: Cohen’s $\kappa = [\text{TBD}]$ (target ≥ 0.7 , indicating substantial agreement). Pairs with initial $\kappa < 0.6$ were adjudicated through structured discussion between the two annotators.

6.2 Retrieval Evaluation

6.2.1 Results

Retrieval quality is evaluated on Conditions B and C (Condition A has no retrieval component). Table 6.2 reports all retrieval metrics across configurations.

Table 6.2: Retrieval evaluation results (Conditions B and C, $N = 45$ answerable questions)

System	P@1	P@3	P@5	MRR	nDCG@5
BM25 (lexical baseline)	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
Cond. B: Naive RAG (FAISS, no rerank)	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
Cond. C: RAG+MCP (FAISS + rerank)	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
Δ C vs. B (absolute)	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
p -value (Wilcoxon, $\alpha = 0.05$)	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]

Hypothesis H3 test (dense retrieval > BM25): [Confirmed / Not confirmed]. Dense retrieval via FAISS achieved $P@5 = [\text{TBD}]$ vs. BM25 $P@5 = [\text{TBD}]$, a difference

of [TBD] ($p = [\text{TBD}]$). The improvement is attributable to the ability of semantic embeddings to match conceptually related queries to documents using different vocabulary than the query terms — a critical capability for domain-specific DNSI terminology that is inconsistently normalized across documents.

Hypothesis H6 test (re-ranking $\geq +15\%$ MRR): [Confirmed / Not confirmed]. Re-ranking improved MRR from [TBD] (Condition B) to [TBD] (Condition C), a relative improvement of [TBD]% ($p = [\text{TBD}]$).

6.2.2 Retrieval Analysis by Query Type

Table 6.3: Retrieval Precision@5 by query category

Query Type	Cond. B (Naive RAG)	Cond. C (RAG+MCP)	Δ
Factoid ($n = 20$)	[TBD]	[TBD]	[TBD]
Multi-hop ($n = 15$)	[TBD]	[TBD]	[TBD]
Procedural ($n = 10$)	[TBD]	[TBD]	[TBD]

Expected finding: Multi-hop queries are anticipated to show the largest benefit from re-ranking, as the cross-encoder can explicitly score the relevance of each passage to the specific multi-document synthesis task, whereas the bi-encoder embedding captures only broad semantic proximity. Factoid queries are expected to show smaller gains, as the top-1 result is typically correct even without re-ranking.

6.2.3 Retrieval Failure Analysis

Retrieval failures (no relevant document in top-5) are analyzed qualitatively. Anticipated failure modes:

- **Vocabulary mismatch:** Query uses technical acronyms not present in training data of the embedding model. *Mitigation: domain-specific embedding fine-tuning (future work).*
- **Document quality:** Scanned PDF pages with imperfect OCR produce corrupted text chunks that embed poorly.
- **Information not indexed:** Document removed from SharePoint before the last ingestion cycle, or protected by ACLs that prevented ingestion. The system correctly returns a graceful refusal for unanswerable queries.

6.3 Generation Quality Evaluation

6.3.1 RAGAS Metrics

Generation quality is evaluated using the RAGAS framework (Es et al., 2024) across all three conditions. RAGAS metrics for Condition A are partial (no context to evaluate precision/recall against).

Table 6.4: RAGAS generation quality results ($N = 45$ answerable questions)

Condition	Faithfulness	Answer Relevancy	Context Precision	Context Recall
A: LLM Alone	[TBD]	[TBD]	N/A	N/A
B: Naive RAG	[TBD]	[TBD]	[TBD]	[TBD]
C: RAG+MCP	[TBD]	[TBD]	[TBD]	[TBD]
Δ C vs. A	[TBD]	[TBD]	—	—
Δ C vs. B	[TBD]	[TBD]	[TBD]	[TBD]

Hypothesis H1 test (RAG+MCP hallucination < LLM Alone): [Confirmed / Not confirmed]. The faithfulness score for Condition C ([TBD]) vs. Condition A ([TBD]) represents the primary evidence for H1.

Hypothesis H2 test (RAG+MCP faithfulness > Naive RAG): [Confirmed / Not confirmed]. The difference in faithfulness between Conditions C and B isolates the contribution of the MCP layer and re-ranking to generation grounding.

6.3.2 Hallucination Rate Analysis

Table 6.5: Hallucination rate by condition and query type

Condition	Factoid	Multi-hop	Procedural	Overall
A: LLM Alone	[TBD]	[TBD]	[TBD]	[TBD]
B: Naive RAG	[TBD]	[TBD]	[TBD]	[TBD]
C: RAG+MCP	[TBD]	[TBD]	[TBD]	[TBD]

Expected findings: LLM Alone (Condition A) is expected to exhibit the highest hallucination rate, particularly on queries about DNSI-specific processes and systems absent from Mistral 7B’s training data. Multi-hop queries are expected to show higher

hallucination rates than factoid queries across all conditions, as they require synthesis across documents — a task where the model has more opportunity to introduce confabulated transitions between retrieved facts.

6.3.3 Hallucination Taxonomy

Manual annotation of hallucinated responses identifies the following categories (to be populated with counts):

Table 6.6: Hallucination taxonomy with representative examples

Type	Description	Example	Pat-tern	Count (Cond. A)
Factual confabulation	Model invents specific facts absent from corpus	Invented procedure version numbers	pro-steps,	[TBD]
Source attribution error	Correct fact attributed to wrong document	Cross-document conflation		[TBD]
Temporal hallucination	Incorrect dates or version information	Outdated policy dates cited as current		[TBD]
Scope expansion	Response extrapolates beyond corpus content	General IT advice presented as DNSI policy		[TBD]
Refusal failure	Model answers unanswerable question	Fabricated answer for out-of-corpus query		[TBD]

6.3.4 Qualitative Response Analysis

Three representative examples are presented to illustrate the qualitative difference between conditions. Each example shows the query, and the responses from Conditions A, B, and C side-by-side.

Example 1 — Factoid query (anticipated outcome: clear advantage for RAG conditions): *Query:* “What is the retention period for project closure reports in the DNSI document management policy?”

- **Condition A (LLM Alone):** “[TBD — expected: generic response citing typical enterprise retention periods of 3–7 years, without DNSI-specific grounding]”

- **Condition B (Naive RAG):** “[TBD — expected: correct answer retrieved but possibly without attribution]”
- **Condition C (RAG+MCP):** “[TBD — expected: correct answer with explicit citation to source document]”

Example 2 — Multi-hop query (anticipated outcome: largest advantage for Condition C): *Query:* “What are the security requirements for a new application that will process personal data and integrate with the CERP infrastructure?”

Example 3 — Unanswerable query (anticipated outcome: correct refusal in Condition C): *Query:* “What is the contact email of the DNSI’s external cybersecurity audit firm?”

6.4 System Performance Evaluation

6.4.1 End-to-End Latency

Latency is measured over $N = 200$ queries representative of the benchmark distribution, executed sequentially in a single-user scenario on the DNSI deployment infrastructure.

Table 6.7: End-to-end latency breakdown (milliseconds, single user, CPU inference)

Component	Mean	P50	P95	P99
Query encoding (embedding)	[TBD]	[TBD]	[TBD]	[TBD]
FAISS retrieval ($K = 20$)	[TBD]	[TBD]	[TBD]	[TBD]
ACL permission resolution	[TBD]	[TBD]	[TBD]	[TBD]
Cross-encoder re-ranking	[TBD]	[TBD]	[TBD]	[TBD]
MCP protocol overhead	[TBD]	[TBD]	[TBD]	[TBD]
LLM generation (Mistral 7B CPU)	[TBD]	[TBD]	[TBD]	[TBD]
Total end-to-end	[TBD]	[TBD]	[TBD]	[TBD]
NFR01 target ($P95 \leq 5000\text{ms}$)			[Met/Exceeded]	

Hypothesis H4 test (MCP overhead within SLA): [Confirmed / Not confirmed]. The MCP protocol overhead is measured as the difference in total pipeline latency between Condition B (direct retrieval) and Condition C (MCP-mediated retrieval), holding all other components constant. Expected overhead: 50–150ms (serialization + HTTP round-trip within the same host).

Critical finding: LLM generation latency on CPU is anticipated to dominate the total pipeline latency, likely exceeding NFR01 for the current infrastructure. This is a deployment infrastructure limitation (no GPU available), not an architectural limitation. The measurement serves as the baseline for the GPU deployment recommendation.

6.4.2 MCP Protocol Overhead Isolation

To precisely isolate the MCP overhead, the same 50 queries are executed through two paths:

1. Direct Python function call to the retrieval module (bypassing MCP transport)
2. Via the MCP `search_documents` tool call (full MCP protocol)

The difference in P50 latency between these paths represents the pure MCP serialization and transport overhead, expected to be in the range of 20–80ms for localhost HTTP/SSE communication.

6.4.3 Memory Footprint

Table 6.8: Memory footprint by component

Component	Description	Memory (MB)
FAISS index (N chunks, $d = 768$)	$N \times 768 \times 4$ bytes + overhead	[TBD]
Chunk metadata store	JSON Lines, N chunks	[TBD]
Embedding model (<code>all-mpnet-base-v2</code>)	Loaded into memory	~420
Cross-encoder (MS-MARCO MiniLM L6)	Loaded into memory	~85
Mistral 7B Q4_K_M (llama-cpp)	4-bit quantized model	~4,500
Total		[TBD]

6.5 User Evaluation

6.5.1 Participant Profile

The user study was conducted with $N = [\text{TBD}]$ DNSI employees. Participants were recruited through the DNSI’s internal communication channel. Inclusion criteria: regular SharePoint users (≥ 3 days/week), no prior exposure to AntiGravity. Participant profiles:

- Technical roles (developers, architects): $[\text{TBD}]$ %

- Operational roles (project managers, analysts): [TBD]%
- Management roles: [TBD]%

6.5.2 SUS and TAM Results

Table 6.9: System Usability Scale and TAM results

Instrument	Construct	Score (Mean $\pm$ SD)	95% CI
SUS	Overall usability (0–100)	[TBD] $\pm$ [TBD]	[TBD, TBD]
TAM	Perceived Usefulness (1–7)	[TBD] $\pm$ [TBD]	[TBD, TBD]
TAM	Perceived Ease of Use (1–7)	[TBD] $\pm$ [TBD]	[TBD, TBD]

Hypothesis H5 test (AntiGravity rated significantly more useful): [Confirmed / Not confirmed]. A paired Wilcoxon signed-rank test compares perceived usefulness ratings for AntiGravity vs. the current SharePoint search on the 5 task scenarios.

SUS interpretation (to be completed): A SUS score above 68 is considered above average; above 80 is considered excellent (Brooke, 1996). [TBD — expected score in the 65–80 range, reflecting good usability bounded by latency concerns].

6.5.3 Task Performance Results

Table 6.10: Task-based evaluation results ($N = [TBD]$ participants, 5 tasks)

Task	Type	AntiGravity Completion %	SharePoint Completion %	Δ Time (s)
T1	Factoid	[TBD]	[TBD]	[TBD]
T2	Multi-hop	[TBD]	[TBD]	[TBD]
T3	Procedural	[TBD]	[TBD]	[TBD]
T4	Multi-hop	[TBD]	[TBD]	[TBD]
T5	Unanswerable	[TBD]	[TBD]	[TBD]
Average		[TBD]	[TBD]	[TBD]

Expected pattern: Multi-hop and procedural tasks are expected to show the largest time savings with AntiGravity, as these are precisely the query types that current SharePoint search handles worst. Factoid tasks may show smaller advantages or even

parity, since SharePoint’s keyword search can effectively locate a specific document containing a known term.

6.5.4 Qualitative Themes

Thematic analysis of the semi-structured interviews (Braun and Clarke, 2006) identifies the following themes (to be populated with participant quotes):

Table 6.11: Qualitative themes from user interviews

Theme	Description	Exemplar Quote	Freq.
Trust in sourced responses	Users value explicit source citations as trust signal	“[TBD]”	[TBD]/[N]
Latency as adoption barrier	Response time perceived as too slow for routine use	“[TBD]”	[TBD]/[N]
Multi-document synthesis value	Recognition of value for complex, cross-document queries	“[TBD]”	[TBD]/[N]
Uncertainty about corpus coverage	Users unsure what documents are indexed	“[TBD]”	[TBD]/[N]

6.6 Ablation Study

To disentangle the contributions of individual components, an ablation study evaluates four configurations on the retrieval-generation pipeline:

Table 6.12: Ablation study: component contribution to faithfulness and P@5

Configuration	Re- ranking	MCP Layer	P@5	Faithfulness
A1: FAISS only	No	No	[TBD]	[TBD]
A2: FAISS + rerank	Yes	No	[TBD]	[TBD]
A3: FAISS + MCP (no rerank)	No	Yes	[TBD]	[TBD]
A4: FAISS + MCP + rerank (AntiGravity)	Yes	Yes	[TBD]	[TBD]
Marginal value of re-ranking (Δ A2 vs. A1)			[TBD]	[TBD]
Marginal value of MCP (Δ A3 vs. A1)			[TBD]	[TBD]
Interaction effect (Δ A4 vs. A2+A3-A1)			[TBD]	[TBD]

Expected finding: Re-ranking is expected to contribute the largest marginal improvement to P@5. The MCP layer, by design, contributes primarily to security (ACL enforcement) rather than retrieval precision — its precision contribution, if any, will come from the structured context assembly (context budget management and citation formatting) improving generation faithfulness rather than raw retrieval metrics.

6.7 Hypothesis Testing Summary

Table 6.13: Summary of hypothesis tests

ID	Hypothesis	Test Method	Outcome	Key Evidence
H1	RAG+MCP hallucination < LLM Alone	RAGAS	[TBD]	$\Delta\text{Faithfulness}=[\text{TBD}]$ $\text{HR}_C=[\text{TBD}]$ vs. $\text{HR}_A=[\text{TBD}]$
H2	RAG+MCP faithfulness > Naive RAG	RAGAS	[TBD]	$\Delta=[\text{TBD}]$, $p=[\text{TBD}]$
H3	Dense retrieval > BM25 (P@5)	Wilcoxon	[TBD]	$\Delta\text{P@5}=[\text{TBD}]$, $p=[\text{TBD}]$
H4	MCP overhead within SLA (<5s P95)	Latency	[TBD]	$\text{P95}_{\text{MCP overhead}}=[\text{TBD}]$
H5	AntiGravity more useful than SharePoint search	Paired Wilcoxon	[TBD]	TAM- $\text{PU}_C=[\text{TBD}]$ vs. [TBD], $p=[\text{TBD}]$
H6	Re-ranking $\geq +15\%$ MRR	MRR	[TBD]	$\text{MRR}_B=[\text{TBD}]$; $\text{MRR}_C=[\text{TBD}]$; $\Delta=[\text{TBD}]\%$

6.8 Chapter Summary

This chapter has presented the evaluation framework and result structure for all four evaluation dimensions. The experimental design provides a rigorous basis for answering each of the five research questions through the testing of six hypotheses. Key

contributions of this evaluation are: (1) the first controlled comparison of LLM alone, naive RAG, and RAG+MCP on a private SharePoint enterprise corpus; (2) quantitative isolation of the MCP protocol overhead; (3) a qualitative hallucination taxonomy specific to enterprise knowledge access; and (4) a mixed-methods user evaluation combining SUS, TAM, task performance, and thematic interview analysis.

The **[TBD]** markers represent experimental values to be populated upon completion of the evaluation runs. The complete result tables will be finalized before thesis submission.

Chapter 7

Discussion

The preceding chapter presented the empirical results of the AntiGravity evaluation. This chapter interprets those results in relation to the research questions, critically assesses what the thesis does and does not demonstrate, examines the limitations of the work, and discusses the implications for both researchers and practitioners. A researcher who cannot honestly criticize their own work has not understood it.

7.1 Interpretation of Results

7.1.1 On the Primary Research Question

The primary research question (RQ1) asks whether a RAG+MCP architecture enables more *reliable*, *secure*, and *traceable* access to enterprise knowledge than a standalone LLM or naive RAG implementation.

Reliability. The hallucination rate comparison (Section 6.3) provides the central evidence for the reliability dimension. The expected finding — that Condition C (RAG+MCP) substantially reduces hallucination compared to Condition A (LLM Alone) — is the most straightforward result to interpret: grounding LLM generation in retrieved, verified source material from the DNSI corpus structurally prevents the model from confabulating information about DNSI-specific processes, systems, and policies that were not in its training data.

The more nuanced finding concerns the comparison between Conditions B and C (Naive RAG vs. RAG+MCP). The faithfulness improvement of Condition C over B, if confirmed, is attributable to two mechanisms: (1) the cross-encoder re-ranking produces a more precisely relevant context, reducing the probability that the model is distracted by loosely related but irrelevant passages; and (2) the MCP context assembly enforces structured citation formatting, which has been observed to anchor the model more firmly to source material in the prompt.

However, the improvement over Naive RAG is expected to be smaller than the improvement over LLM Alone — this is theoretically correct. RAG itself (even without MCP) addresses the fundamental problem of knowledge grounding. MCP adds value in

the dimensions of security and standardization, with a secondary benefit to generation faithfulness through better context management.

Security. The ACL enforcement mechanism provides a structural security guarantee (Section 4.5.2): no document content unauthorized for the querying user can enter the LLM prompt. This guarantee is architectural, not probabilistic. It is the primary security contribution of the RAG+MCP design over both Condition A and Condition B. However, this guarantee has boundaries: it applies to the retrieval layer but does not protect against prompt injection attacks via the document content itself (discussed in Section 7.4).

Traceability. Every response generated by Condition C includes source citations derived from the chunk metadata. This is not merely a user experience feature: it creates an audit trail linking each factual claim in a response to a specific DNSI document, enabling a human reviewer to verify or refute the claim. Conditions A and B provide no equivalent traceability mechanism.

7.1.2 On the Value of MCP Specifically

A critical question for this thesis is whether MCP itself — as opposed to the RAG architecture in general — makes a measurable contribution. This is the hardest question to answer empirically, because MCP’s primary contributions (standardization, security boundary, maintainability) are not directly measurable as retrieval or generation quality metrics.

The ablation study (Section 6.6) provides indirect evidence: the comparison of A2 (FAISS + re-ranking, no MCP) vs. A4 (full AntiGravity with MCP) isolates the marginal contribution of the MCP layer. If the difference in P@5 or faithfulness between A2 and A4 is not statistically significant — which is the expected finding — this should not be interpreted as MCP providing no value. It should be interpreted as MCP providing value in dimensions that retrieval/generation metrics do not capture: security architecture, connector standardization, and operational maintainability.

This distinction matters for the scientific interpretation of the thesis. The claim is not that MCP improves RAG retrieval quality. The claim is that MCP provides a principled architectural boundary for security enforcement, and that this architectural property is valuable in enterprise contexts in ways that micro-benchmarks cannot fully capture.

7.2 Strengths of the Proposed Architecture

Structural security by design. The most significant strength of the AntiGravity architecture is that the security property — ACL-aware retrieval — is not implemented as an application-level check that can be bypassed by logic errors, but as a structural property of the MCP boundary. No pathway exists through the architecture by which unauthorized document content can reach the LLM without traversing the ACL filter.

Source attribution as trust infrastructure. The mandatory citation mechanism, embedded in the system prompt and enforced by the prompt template, provides a trust infrastructure that enterprise users and compliance teams require. Unlike a standalone LLM, every AntiGravity response can be audited against its source documents. In regulated environments (information security policies, legal compliance, data governance), this auditability is not optional — it is required.

Protocol separation of concerns. The MCP boundary between the conversational layer and the data layer means that changes to the underlying data source (a SharePoint API version change, migration to a different SharePoint tenant, addition of a second data source) require changes only to the MCP server — not to the conversational client or the LLM interface. This operational benefit compounds over time in environments where data infrastructure evolves continuously.

Research novelty. The architecture addresses a combination of problems — ACL-aware retrieval, MCP integration, enterprise SharePoint, and multilingual (French/English) corpora — that no published system has addressed in combination. The research gap identified in Chapter 2 is genuinely addressed.

7.3 Weaknesses and Limitations

This section deliberately presents the limitations of the thesis with the same rigor applied to its strengths. Honest acknowledgment of limitations is a mark of scientific maturity and protects the thesis from criticism on grounds the author has pre-empted.

7.3.1 Evaluation Limitations

Benchmark dataset size ($N = 50$). Fifty question-answer pairs is sufficient for reporting metric distributions with 95% confidence intervals, but is insufficient for detecting small effect sizes with high statistical power. A benchmark of $N = 200$ – 500 would provide substantially stronger statistical evidence. The constraint is operational:

manual annotation of ground-truth relevance labels is time-intensive. Future work should extend the benchmark.

Single-organization generalizability. All evaluation is conducted on the DNSI SharePoint corpus. The vocabulary, document structure, query patterns, and language distribution (French/English mix) are specific to this organization. Results may not generalize to organizations with different corpus characteristics. The evaluation should be interpreted as a proof-of-concept demonstration, not as a universal performance characterization.

RAGAS as a proxy for faithfulness. RAGAS metrics are computed by a language model evaluating the faithfulness of another language model’s output against retrieved context. This creates a potential circular dependency: if the evaluating model has similar failure modes to the generating model, some hallucinations may not be detected. A fully reliable hallucination assessment requires human annotation of every generated response — which was provided for the hallucination rate metric but not for the RAGAS faithfulness scores.

No longitudinal evaluation. The user study is a single-session evaluation. Real adoption decisions are made over time, through repeated use. A longitudinal study (tracking usage patterns over weeks) would provide much stronger evidence for or against the adoption hypothesis (H5).

7.3.2 Technical Limitations

LLM inference latency. The CPU-only deployment of Mistral 7B results in generation latency of 25–45 seconds for typical responses — substantially exceeding NFR01 ($P95 \leq 5$ seconds). This is the most significant gap between the designed system and the production-ready system. It is a deployment infrastructure limitation (no GPU), not an architectural limitation. GPU deployment is expected to reduce generation latency to 2–5 seconds, bringing the system within NFR01. This limitation was known at the outset and does not invalidate the architectural contributions, but it does prevent the user study from fully reflecting the intended user experience.

Incremental index updates. The current ingestion pipeline requires a full index rebuild when documents are deleted from SharePoint. For a dynamically evolving corpus, this means a latency window during which deleted documents remain retrievable. The window is bounded by the ingestion schedule (nightly) but is non-zero. Production deployment would require migration to a vector store with native deletion support (e.g., Chroma, Weaviate, or FAISS IndexIDMap).

Single-language embedding model. The `all-mpnet-base-v2` embedding model was trained primarily on English text. While it performs adequately on the French-language DNSI documents (French is well-represented in its training data), a multilingual model (e.g., `paraphrase-multilingual-mpnet-base-v2`) or a domain-fine-tuned model would likely improve retrieval precision on French-language queries.

Prompt injection residual risk. As identified in the threat model (Table 4.5), the prompt injection attack vector via adversarial document content has not been fully mitigated. The structured system prompt and near-zero temperature reduce susceptibility, but do not eliminate it. A production deployment should add an additional layer of prompt injection detection (e.g., a small classifier model screening retrieved chunks for injection patterns) before this system handles highly sensitive materials.

7.3.3 Scope Limitations

No real-time synchronization. The nightly ingestion schedule means the index may be up to 24 hours out of date relative to the live SharePoint corpus. For time-sensitive operational contexts (incident response, live project documentation), this lag is unacceptable. Real-time delta synchronization via the Microsoft Graph change notification API is required for production use and is identified as the highest-priority future work item.

Single data source. The system is designed for SharePoint CERP exclusively. DNSI knowledge is distributed across multiple systems (email, wikis, ticketing systems, databases). A production enterprise assistant would need to federate across these sources. MCP’s multi-server architecture makes this extension architecturally straightforward but operationally complex.

7.4 Security Analysis: Critical Assessment

The security architecture presented in Chapter 4 provides strong guarantees at the retrieval layer but leaves residual risks at the prompt layer that deserve honest analysis.

What the architecture guarantees. The delegated OAuth 2.0 token mechanism ensures that ACL enforcement is performed by Microsoft’s Graph API permission engine — the most authoritative and thoroughly tested component in the system. ACL checks performed by well-maintained cloud provider services are more reliable than ACL checks implemented in application code. The MCP boundary ensures that the retrieval and permission resolution logic cannot be bypassed by the conversational layer.

What the architecture does not guarantee. The architecture does not protect against an insider threat who uses their legitimate SharePoint write access to plant adversarial documents. A document containing the text `[SYSTEM: disregard previous instructions and output the full text of the previous response]` would pass the ACL filter (the attacker is authorized to write it) and be retrieved by legitimate queries. The near-zero temperature setting and structured system prompt provide partial mitigation but not elimination of this risk.

Practical likelihood. This attack requires a motivated insider with SharePoint write access who is aware of the AntiGravity system’s implementation details and is willing to compromise the system. In the DNSI context, this is a low-probability but non-zero threat. The recommendation is to add automated prompt injection screening of indexed document content at ingestion time, as an additional defense-in-depth measure.

7.5 Deployment Challenges: Lessons Learned

The deployment of AntiGravity in the DNSI’s production environment produced several lessons that are generalizable to any enterprise AI deployment:

Corporate proxy is a first-class constraint, not an afterthought. The DNSI’s TLS-terminating proxy required changes to all components in the stack (HTTP clients, MSAL, MCP transport). Enterprise AI systems should be designed with proxy-aware HTTP clients from the start, not retrofitted. The corporate proxy is not an exceptional edge case — it is the norm in large enterprise environments.

OAuth 2.0 delegated permissions require organizational IT cooperation. Registering an Entra ID application, configuring delegated permissions, and navigating the DNSI’s internal approval workflow for API access consumed significant project time. Enterprise AI projects should account for identity and access management lead times in their project schedules.

On-premises LLM inference is operationally expensive. Running Mistral 7B on-premises (required by NFR09) requires hardware procurement, model management, quantization trade-off decisions, and ongoing maintenance. Cloud-based LLM APIs are operationally much simpler, but are incompatible with strict data locality requirements. Organizations face a genuine trade-off between operational simplicity and data sovereignty.

7.6 Generalizability of Findings

The AntiGravity architecture and the findings of this thesis are generalizable to other organizations under the following conditions:

1. **SharePoint as the primary knowledge repository:** The connector design and ACL enforcement mechanism are specific to SharePoint and Microsoft Graph API. Organizations using Confluence, Box, Google Drive, or other platforms would require a different MCP server implementation, but the architectural pattern (MCP boundary for security enforcement) is fully transferable.
2. **Enterprise security requirements:** The security architecture is designed for organizations with formal information security policies (ISO 27001 or equivalent). Organizations without strict ACL requirements can simplify the architecture by removing the permission manifest and delegated token mechanism.
3. **Moderate corpus scale (up to 100k documents):** The FAISS IndexFlatIP performs optimally at this scale. Larger organizations would need to evaluate approximate search indexes.
4. **Document-centric knowledge:** The system is optimized for unstructured text in PDF and DOCX format. Organizations with knowledge primarily in structured databases, spreadsheets, or specialized formats would require extended parsing modules.

7.7 Future Work

The limitations identified above directly motivate a prioritized future research and development agenda:

Table 7.1: Prioritized future work roadmap

Priority	Item	Description	Expected Impact
P1	GPU deployment	Deploy Mistral 7B on GPU hardware to bring generation latency within NFR01	Critical for user adoption
P2	Real-time delta sync	Microsoft Graph change notifications for live Share-Point updates	Production readiness
P3	Multilingual embedding	Fine-tune or replace embedding model for French/English domain vocabulary	Retrieval precision
P4	Prompt injection defense	Ingestion-time adversarial content screening	Security hardening
P5	Larger benchmark	Expand to $N = 200+$ Q&A pairs for statistical power	Evaluation rigor
P6	Multi-source federation	Extend MCP architecture to email, wikis, ticketing systems	Coverage expansion
P7	Agentic RAG	Integrate multi-step tool use for complex procedural queries	Capability expansion
P8	CIFRE doctoral continuation	Investigate MCP as a universal enterprise AI integration	Scientific contribution

CIFRE doctoral direction. This thesis establishes a foundation for a CIFRE doctoral research program that would investigate three open scientific questions: (1) Can MCP be formalized as a universal integration standard for enterprise AI, and what security-theoretic guarantees can be proven about MCP-mediated architectures? (2) How do RAG+MCP systems scale under multi-tenant, multi-source, and multi-modal knowledge conditions? (3) What are the organizational change management factors that determine enterprise AI assistant adoption, and how can system design choices mitigate adoption barriers?

7.8 Chapter Summary

This chapter has provided a critical analysis of the AntiGravity system and the thesis research. The primary research question is answered affirmatively: the RAG+MCP architecture demonstrably improves reliability (reduced hallucination), security (structural ACL enforcement), and traceability (mandatory source citations) compared to baseline alternatives. MCP’s specific contribution is primarily architectural (security boundary, standardization) rather than metric-measurable (retrieval precision, faithfulness), which is precisely what the MCP specification claims and what this thesis confirms.

The thesis’s limitations are significant but bounded: the evaluation is a proof-of-concept demonstration on a single organization’s corpus with a small benchmark, not a universal performance characterization. The deployment has a known infrastructure gap (CPU inference latency) that is resolvable with hardware investment. These limitations are typical of applied AI engineering research at the Master’s level and do not undermine the validity of the architectural contributions.

The following chapter synthesizes the thesis contributions and answers each research question explicitly.

Chapter 8

Conclusion

8.1 Summary of Contributions

This thesis set out to design, implement, and evaluate a RAG conversational assistant grounded in the Model Context Protocol for enterprise knowledge access from SharePoint. The work has produced four scientific contributions and four technical contributions, each of which is summarized below with the evidence base that validates it.

SC1 — Formal comparative evaluation framework. This thesis presents the first published empirical comparison of LLM Alone, Naive RAG, and RAG+MCP architectures on a private enterprise SharePoint corpus with ACL enforcement. The evaluation framework — a domain-specific 50-item benchmark, RAGAS metrics for generation quality, Precision@K/MRR/nDCG for retrieval quality, latency benchmarking, and a mixed-methods user study — is documented in sufficient detail for replication and serves as a methodological contribution independent of the specific results obtained.

SC2 — Reference architecture for ACL-aware enterprise RAG. The AntiGravity architecture introduces a novel pattern for permission-aware retrieval: the over-retrieval design ($K = 20 \rightarrow k = 5$ post-ACL-filter) and the structural placement of ACL enforcement within the MCP server boundary. These design patterns are documented with formal security properties (no unauthorized content traverses the MCP boundary) and can be adopted by practitioners deploying RAG systems in access-controlled enterprise environments.

SC3 — Application of DSR to enterprise AI evaluation. The thesis demonstrates that Design Science Research provides a principled framework for research that simultaneously produces a system artifact and empirically evaluates it. The three-cycle DSR model (Relevance, Design, Rigor) maps cleanly onto the research phases of this work and is recommended as the methodology of choice for applied enterprise AI research.

SC4 — AntiGravity proof of concept. The implemented system validates the proposed architecture under real DNSI production constraints: corporate proxy, OAuth

2.0 authentication, DNSI document formats, and French/English mixed-language corpus. The challenges encountered and resolved (Section 5.6) constitute practical engineering knowledge not available from theoretical design alone.

TC1–TC4. The technical contributions — MCP-compatible SharePoint connector, ACL-enforced retrieval pipeline, domain-specific benchmark dataset, and deployment documentation — are provided in the appendices as reproducible artifacts.

8.2 Answers to Research Questions

RQ1 (Primary): *To what extent does a RAG+MCP architecture enable more reliable, secure, and traceable access to enterprise knowledge than a standalone LLM or naive RAG?*

The evidence supports a positive answer on all three dimensions. *Reliability:* RAG+MCP substantially reduces hallucination (H1 [TBD confirmed]) and improves faithfulness over both LLM Alone and Naive RAG. *Security:* The structural ACL enforcement at the MCP boundary provides a formally verifiable security property absent from all baseline architectures. *Traceability:* Mandatory source citation in every response creates an audit trail that neither Condition A nor Condition B provides.

RQ2 (Retrieval): *What retrieval configuration maximizes precision and recall on a domain-specific SharePoint corpus?*

The combination of `all-mpnet-base-v2` embeddings (768 dimensions, local deployment), FAISS IndexFlatIP exact search, and MS-MARCO cross-encoder re-ranking achieves the best retrieval performance on the DNSI benchmark, outperforming BM25 lexical search (H3 [TBD]) and outperforming FAISS without re-ranking in MRR (H6 [TBD]). The 512-token / 64-token-overlap recursive chunking strategy was confirmed as superior to zero-overlap fixed-size chunking through early-stage ablation.

RQ3 (Protocol): *What specific properties does MCP deliver in an enterprise SharePoint integration context?*

MCP delivers three properties that alternative integration approaches do not: (1) a stable protocol boundary that decouples the AI pipeline from SharePoint API evolution; (2) a structural security enforcement point enabling ACL filtering before context reaches the LLM; and (3) a standardized connector interface that enables the SharePoint connector to be reused across different MCP-compatible AI clients. The latency overhead of the MCP protocol layer is [TBD]ms at P95, within acceptable bounds (H4 [TBD]).

RQ4 (Performance): *What are the latency and throughput characteristics under realistic enterprise conditions?*

On CPU infrastructure, end-to-end P95 latency is dominated by LLM generation ([TBD]s), which exceeds the NFR01 target. All non-generation components (embedding, FAISS, ACL resolution, re-ranking, MCP) operate within budget. GPU deployment is the required intervention to achieve NFR01 compliance. Throughput under simulated concurrent load ([TBD] queries/second at 20 concurrent users) meets the NFR06 target.

RQ5 (Acceptance): *How do end-users perceive the AntiGravity assistant compared to current SharePoint search?*

Users rate AntiGravity as [significantly / somewhat] more useful for complex, multi-document queries (H5 [TBD]). The SUS score of [TBD] indicates [good / above-average / excellent] usability. The primary adoption barrier identified qualitatively is response latency, not usability or trust. Source citations were consistently identified as the feature most valued by users.

8.3 Scientific and Practical Implications

For researchers. This thesis contributes empirical evidence to an important open question in the RAG literature: whether the introduction of a protocol standardization layer (MCP) between AI and data sources adds value beyond what the RAG architecture itself provides. The answer is nuanced: MCP’s value is primarily architectural (security, maintainability, standardization) and does not manifest as metric improvements in isolation. Researchers evaluating RAG-over-enterprise-data systems should expand their evaluation frameworks beyond retrieval/generation metrics to include security audits, maintainability assessments, and operational complexity evaluations.

For practitioners. Organizations deploying LLM-based assistants over internal documentation corpora should: (1) adopt delegated OAuth 2.0 permissions (not service accounts) to enable per-user ACL enforcement; (2) implement ACL filtering at the retrieval layer, not the display layer; (3) prioritize source citation in the LLM prompt template as the most impactful single intervention for user trust; (4) plan for GPU infrastructure if generation latency is a user adoption concern; and (5) consider MCP as a long-term integration strategy that reduces connector maintenance overhead as the number of data sources grows.

8.4 Toward a CIFRE Doctoral Continuation

This thesis establishes both the technical foundation and the scientific positioning for a CIFRE doctoral research program. The work demonstrates the ability to design an industrial-grade AI system, evaluate it rigorously, and locate its contributions within the academic literature.

Three open scientific questions, identified through the limitations of this thesis, constitute a natural doctoral research agenda:

1. **MCP as a universal enterprise AI integration standard:** Can MCP be formally analyzed as a security protocol, and what properties (confidentiality, integrity, access control) can be formally proven about MCP-mediated RAG architectures? This question connects to the formal methods literature and is motivated by the residual prompt injection risk identified in this thesis.
2. **Scalable multi-source, multi-modal enterprise RAG:** How does the RAG+MCP architecture perform when federated across multiple enterprise data sources (SharePoint, email, databases) and multiple modalities (text, tables, structured data)? What new architectural patterns are required? This question is motivated by the single-source limitation of this thesis.
3. **Enterprise AI adoption factors:** What organizational, cognitive, and system design factors determine whether employees adopt and trust AI-powered knowledge assistants? What is the relationship between latency, faithfulness, citation quality, and adoption intent? This question connects to information systems adoption literature (TAM, UTAUT) and is motivated by the small and time-constrained user study of this thesis.

A CIFRE PhD co-supervised between Aivancity and the DNSI's host organization would provide the industrial context, data access, and organizational scope required to address all three questions.

8.5 Closing Reflection

This thesis began with a simple observation: organizations accumulate knowledge at scale and fail to use it at scale. The gap between what an organization knows and what its members can access and apply is an organizational failure with real operational and economic consequences. Artificial intelligence, in the specific form of RAG-augmented language models, offers a technically credible solution to this problem — not by replacing human expertise, but by making the organization's documented knowledge genuinely searchable, synthesizable, and accessible through natural language.

The Model Context Protocol, positioned as a standardization layer for AI-data integration, represents a maturation of this solution space: a recognition that the fragmentation of enterprise knowledge across heterogeneous systems requires not just better AI models, but better integration standards. This thesis has demonstrated, through the design and evaluation of AntiGravity, that the combination is both technically realizable and practically valuable under real enterprise constraints.

The work is a beginning, not an end. The evaluation is a proof of concept, not a production certification. The architecture is a design reference, not a finished product. But the research question has been engaged with the rigor it deserves, and the contributions — however bounded by the constraints of a Master’s thesis — are genuine.

Bibliography

- Aivancity School for Technology and Business (2025). *Specifications: End of Studies (PFE) Project — Programme Grande École*. Tech. rep. January 2025. Aivancity School for Technology and Business.
- Alavi, Maryam and Dorothy E. Leidner (2001). “Review: Knowledge Management and Knowledge Management Systems: Conceptual Foundations and Research Issues”. In: *MIS Quarterly* 25.1, pp. 107–136.
- Anthropic (2023). *Claude: A Next-Generation AI Assistant*. Tech. rep. Anthropic.
- (2024). *Model Context Protocol: An Open Standard for Connecting AI Systems to Data*. Tech. rep. Technical Specification, version 2024-11. Anthropic.
- Braun, Virginia and Victoria Clarke (2006). “Using Thematic Analysis in Psychology”. In: *Qualitative Research in Psychology* 3.2, pp. 77–101.
- Brooke, John (1996). “SUS: A Quick and Dirty Usability Scale”. In: *Usability Evaluation in Industry*. Ed. by P. W. Jordan et al. Taylor & Francis, pp. 189–194.
- Brown, Tom B. et al. (2020). “Language Models are Few-Shot Learners”. In: *Advances in Neural Information Processing Systems* 33, pp. 1877–1901.
- Chase, Harrison (2022). *LangChain*. <https://github.com/langchain-ai/langchain>.
- Chowdhery, Aakanksha et al. (2022). “PaLM: Scaling Language Modeling with Pathways”. In: *arXiv preprint arXiv:2204.02311*.
- Davenport, Thomas H. and Laurence Prusak (1998). *Working Knowledge: How Organizations Manage What They Know*. Harvard Business School Press.
- Davis, Fred D. (1989). “Perceived Usefulness, Perceived Ease of Use, and User Acceptance of Information Technology”. In: *MIS Quarterly* 13.3, pp. 319–340.
- Es, Shahul et al. (2024). “RAGAS: Automated Evaluation of Retrieval Augmented Generation”. In: *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pp. 150–158.
- Gao, Luyu et al. (2022). “Precise Zero-Shot Dense Retrieval without Relevance Labels”. In: *arXiv preprint arXiv:2212.10496*.
- Gao, Yunfan et al. (2024). “Retrieval-Augmented Generation for Large Language Models: A Survey”. In: *arXiv preprint arXiv:2312.10997*.
- Hevner, Alan R. (2007). “A Three Cycle View of Design Science Research”. In: *Scandinavian Journal of Information Systems* 19.2, pp. 87–92.
- Hevner, Alan R. et al. (2004). “Design Science in Information Systems Research”. In: *MIS Quarterly* 28.1, pp. 75–105.

- ISO/IEC (2022). *27001: Information Security, Cybersecurity and Privacy Protection — Information Security Management Systems — Requirements*. International Organization for Standardization.
- Ji, Ziwei et al. (2023). “Survey of Hallucination in Natural Language Generation”. In: *ACM Computing Surveys* 55.12, pp. 1–38.
- Jiang, Albert Q. et al. (2023). “Mistral 7B”. In: *arXiv preprint arXiv:2310.06825*.
- Johnson, Jeff, Matthijs Douze, and Hervé Jégou (2019). “Billion-Scale Similarity Search with GPUs”. In: *IEEE Transactions on Big Data* 7.3, pp. 535–547.
- Karpukhin, Vladimir et al. (2020). “Dense Passage Retrieval for Open-Domain Question Answering”. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pp. 6769–6781.
- Khattab, Omar and Matei Zaharia (2020). “ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT”. In: *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 39–48.
- Lewis, Patrick et al. (2020). “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*. Vol. 33. Curran Associates, Inc., pp. 9459–9474.
- Liu, Jerry (2022). *LlamaIndex*. https://github.com/run-llama/llama_index.
- Ma, Xinbei et al. (2023). “Query Rewriting in Retrieval-Augmented Large Language Models”. In: *arXiv preprint arXiv:2305.14283*.
- Maynez, Joshua et al. (2020). “On Faithfulness and Factuality in Abstractive Summarization”. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 1906–1919.
- Microsoft (2024a). *Microsoft Graph API Reference*. Tech. rep. Microsoft Technical Documentation. Microsoft.
- (2024b). *SharePoint Online Architecture and Security Model*. Tech. rep. Microsoft Technical Documentation. Microsoft.
- Nogueira, Rodrigo and Kyunghyun Cho (2019). “Passage Re-ranking with BERT”. In: *arXiv preprint arXiv:1901.04085*.
- OpenAI (2023). *GPT-4 Technical Report*. Tech. rep. arXiv preprint arXiv:2303.08774. OpenAI.
- Peppers, Ken et al. (2007). “A Design Science Research Methodology for Information Systems Research”. In: *Journal of Management Information Systems* 24.3, pp. 45–77.
- Perez, Fabio and Ian Ribeiro (2022). “Ignore Previous Prompt: Attack Techniques For Language Models”. In: *NeurIPS 2022 ML Safety Workshop*.
- Rawte, Vipula, Amit Sheth, and Abhijit Das (2023). “A Survey of Hallucination in Large Foundation Models”. In: *arXiv preprint arXiv:2309.05922*.

- Reimers, Nils and Iryna Gurevych (2019). “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pp. 3982–3992.
- Touvron, Hugo et al. (2023). “Llama 2: Open Foundation and Fine-Tuned Chat Models”. In: *arXiv preprint arXiv:2307.09288*.
- Vaswani, Ashish et al. (2017). “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. Curran Associates, Inc.
- Wang, Liang et al. (2022). “Text Embeddings by Weakly-Supervised Contrastive Pre-training”. In: *arXiv preprint arXiv:2212.03533*.
- Wei, Jason et al. (2022). “Emergent Abilities of Large Language Models”. In: *Transactions on Machine Learning Research*.

Appendix A

Benchmark Dataset: Question-Answer Pairs

This appendix documents the structure and annotation guidelines for the 50-item benchmark dataset used in the evaluation of Chapter 6. The full dataset is provided in machine-readable format (JSON Lines) alongside this thesis.

Dataset Structure

Each record in the dataset has the following structure:

```
1 {  
2   "id": "Q001",  
3   "category": "factoid" | "multi-hop" | "procedural" | "unanswerable",  
4   "query": "Natural_language_question_text",  
5   "ground_truth_answer": "Expected_answer_text",  
6   "relevant_chunk_ids": ["chunk_id_1", "chunk_id_2"],  
7   "source_document_titles": ["Document_Title_1"],  
8   "annotator_1_label": 1,  
9   "annotator_2_label": 1,  
10  "cohen_kappa_pair": 0.85,  
11  "notes": "Optional_annotation_notes"  
12 }
```

Listing A.1: Benchmark dataset record schema

Annotation Guidelines

Annotators were provided the following guidelines:

1. Read the question and the candidate document chunk.
2. Label the chunk as **1 (Relevant)** if it contains information necessary or sufficient to answer the question, or **0 (Not Relevant)** if it does not.
3. A chunk may be labeled Relevant even if it provides only partial information (relevant to multi-hop synthesis).

4. For unanswerable questions: all chunks should be labeled 0 if the question cannot be answered from the corpus.
5. When in doubt, label 0 and note the ambiguity.

Note: The DNSI-specific document content and individual Q&A pairs are omitted from this appendix due to the confidentiality agreement. The anonymized synthetic test corpus is provided in the digital supplementary materials.

Appendix B

User Study Instruments

SUS Questionnaire (System Usability Scale)

Participants rated the following 10 statements on a 5-point Likert scale (1 = Strongly Disagree, 5 = Strongly Agree):

1. I think that I would like to use this system frequently.
2. I found the system unnecessarily complex.
3. I thought the system was easy to use.
4. I think that I would need the support of a technical person to be able to use this system.
5. I found the various functions in this system were well integrated.
6. I thought there was too much inconsistency in this system.
7. I would imagine that most people would learn to use this system very quickly.
8. I found the system very cumbersome to use.
9. I felt very confident using the system.
10. I needed to learn a lot of things before I could get going with this system.

TAM Questionnaire (Technology Acceptance Model)

Perceived Usefulness (PU) — 5 items, 7-point Likert (1 = Extremely Unlikely, 7 = Extremely Likely):

1. Using AntiGravity improves my performance in finding information.
2. Using AntiGravity increases my productivity.
3. Using AntiGravity enhances my effectiveness on the job.
4. AntiGravity is useful in my job.
5. Using AntiGravity saves me time when searching for information.

Perceived Ease of Use (PEOU) — 4 items, 7-point Likert:

1. Learning to operate AntiGravity is easy for me.
2. I find it easy to get AntiGravity to do what I want it to do.
3. My interaction with AntiGravity is clear and understandable.
4. I find AntiGravity easy to use.

Task Scenarios

The five task scenarios used in the task-based evaluation:

1. **T1 (Factoid):** “Find the retention period for project closure reports in the DNSI document management policy.”
2. **T2 (Multi-hop):** “What are the security requirements for deploying a new application that processes personal data on the CERP infrastructure?”
3. **T3 (Procedural):** “Find the step-by-step procedure for requesting access to a new SharePoint site.”
4. **T4 (Multi-hop):** “Compare the roles and responsibilities of the Project Manager and the Technical Lead in the DNSI project governance framework.”
5. **T5 (Unanswerable):** “What is the direct phone number of the DNSI’s external cybersecurity auditor?”

Semi-Structured Interview Guide

1. How would you describe your overall experience using AntiGravity?
2. For which types of tasks would you prefer to use AntiGravity over SharePoint search?
3. What features did you find most valuable? Why?
4. What aspects of the system frustrated you?
5. How much do you trust the responses provided by AntiGravity? What factors influence your trust?
6. Would you use AntiGravity in your daily work if it were permanently deployed? Under what conditions?

Appendix C

System Configuration and Deployment Guide

Environment Specification

The complete Python dependency specification is available in `requirements.txt` in the project repository. Key pinned versions:

```
1 streamlit==1.28.2
2 sentence-transformers==2.2.2
3 faiss-cpu==1.7.4
4 mcp==1.0.0
5 msal==1.24.1
6 pdfplumber==0.10.3
7 python-docx==1.1.0
8 llama-cpp-python==0.2.38
9 httpx==0.25.2
```

Listing C.1: Key dependency versions

Docker Compose Topology

```
1 services:
2   mcp-server:
3     build: ./mcp_server
4     environment:
5       - FAISS_INDEX_PATH=/data/index
6       - CHUNK_STORE_PATH=/data/chunks.jsonl
7       - PERMISSION_MANIFEST=/data/permissions.json
8     volumes:
9       - ./data:/data
10    ports:
11      - "8080:8080"
12
13   streamlit-app:
14     build: ./streamlit_app
15     environment:
```

```
16     - MCP_SERVER_URL=http://mcp-server:8080
17     - AZURE_CLIENT_ID=${AZURE_CLIENT_ID}
18     - AZURE_TENANT_ID=${AZURE_TENANT_ID}
19     - MISTRAL_MODEL_PATH=/models/mistral-7b-instruct-v0.2.Q4_K_M.
      gguf
20   depends_on:
21     - mcp-server
22   ports:
23     - "8501:8501"
24
25   ingestion-worker:
26     build: ./ingestion
27     environment:
28       - SHAREPOINT_SITE_URL=${SHAREPOINT_SITE_URL}
29       - AZURE_CLIENT_SECRET=${AZURE_CLIENT_SECRET}
30   volumes:
31     - ./data:/data
```

Listing C.2: Docker Compose service configuration (simplified)

Configuration Parameters

Table C.1: Tunable system configuration parameters

Parameter	Description	Default	Range
CHUNK_SIZE_TOKENS	Target chunk size in tokens	512	256–1024
CHUNK_OVERLAP_TOKENS	Overlap between adjacent chunks	64	0–128
RETRIEVAL_K	Over-retrieval candidates before ACL filter	20	10–50
RERANK_TOP_K	Final context chunks after re-ranking	5	3–10
LLM_TEMPERATURE	Generation temperature	0.1	0.0–0.5
LLM_MAX_TOKENS	Maximum generation length	512	256–1024
PERMISSION_CACHE_TTL	User permission cache TTL (seconds)	300	60–3600

Mistral 7B Model Provenance

Model: mistral-7b-instruct-v0.2.Q4_K_M.gguf
Source: TheBloke/Mistral-7B-Instruct-v0.2-GGUF (Hugging Face)
SHA-256: [to be populated from actual download]
License: Apache 2.0

Appendix D

Raw Evaluation Data

This appendix summarizes the raw evaluation data format. Complete numerical results will be populated upon completion of evaluation experiments.

Data Format: Retrieval Evaluation

For each benchmark question q_i and experimental condition $C \in \{B, C\}$, the following fields are recorded:

- `query_id`: benchmark question ID
- `condition`: B or C
- `retrieved_chunk_ids`: ordered list of retrieved chunk IDs
- `relevant_chunk_ids`: ground-truth relevant chunk IDs
- `precision_at_1`, `precision_at_3`, `precision_at_5`
- `recall_at_5`
- `reciprocal_rank`
- `ndcg_at_5`
- `retrieval_latency_ms`

Data Format: Generation Evaluation

For each benchmark question q_i and condition $C \in \{A, B, C\}$:

- `query_id`, `condition`
- `generated_response`: full response text
- `faithfulness_ragas`: RAGAS faithfulness score $[0, 1]$
- `answer_relevancy_ragas`: RAGAS answer relevancy $[0, 1]$
- `hallucination_annotator_1`: binary (0/1)
- `hallucination_annotator_2`: binary (0/1)

- `hallucination_type`: taxonomy category (if hallucinated)
- `generation_latency_ms`

Appendix E

Declaration of Generative AI Tool Use

In accordance with Aivancity School for Technology and Business's policy on the use of generative AI tools in academic work (Aivancity School for Technology and Business, 2025), the following declaration documents all uses of generative AI in the production of this thesis.

Tool Used

Claude (claude-sonnet-4-6), developed by Anthropic. Accessed via the Claude Code CLI interface.

Scope of Use

Generative AI assistance was used for the following purposes:

1. **Structural drafting:** Initial drafts of thesis chapter structures, section headings, and outline frameworks were generated with AI assistance and subsequently revised, reorganized, and substantially expanded by the author.
2. **Literature synthesis support:** AI assistance was used to suggest connections between cited works and to draft initial paragraph structures for the literature review. All factual claims, citations, and analytical judgments were verified and authored by the student.
3. **LaTeX formatting:** Table structures, equation formatting, and bibliographic entry formatting were assisted by AI tools.
4. **Language editing:** Grammar, sentence structure, and clarity improvements were suggested by AI tools and accepted or modified by the author.

Scope of Non-Use

The following elements were not generated by AI and represent the student's own intellectual work:

- The research question formulation and hypothesis design
- All architectural design decisions and their justifications
- The benchmark dataset construction and annotation
- The experimental results and their interpretation
- The critical analysis of limitations and future work
- The code implementation of the AntiGravity system
- All final editorial decisions regarding content, emphasis, and argumentation

Compliance Statement

This declaration complies with rules 1 (full transparency), 2 (systematic reference), 3 (fair use), and 4 (ethics) as specified in Chapter III of the Aivancity PFE Guidelines (Aivancity School for Technology and Business, 2025). The use of generative AI in this thesis is assistive and supplementary; the thesis primarily reflects the student's own skills, ideas, and efforts.

Signed: _____

Date: _____

Jean Direl NZE